

第 5 卷 图论与树论

考试速查：主查图树模板：统一 1-index 建图、无权 BFS、非负权 Dijkstra、小图 Floyd、拓扑、最小生成树、强连通、LCA 和树上 DP。

Contents

第 5 卷 图论与树论	2
考试速查定位	2
使用顺序	2
GRAPH-00 标准 Graph 与建图场景	2
图类型与模块路由协议	4
GRAPH-01 DFS/BFS 连通与遍历	5
GRAPH-02 无权最短路 BFS	8
GRAPH-03 Dijkstra、路径恢复、多源最短路	10
GRAPH-04 Floyd、Bellman-Ford 与 SPFA	13
GRAPH-05 拓扑排序与 DAG DP	16
GRAPH-06 DSU 与 Kruskal 最小生成树	19
GRAPH-07 二分图染色与二分图匹配	22
GRAPH-08 有向图强连通分量 SCC	24
GRAPH-09 树上 DFS、距离与 LCA	28
GRAPH-10 Dinic 最大流低优先级补充	31
GRAPH-11: 图/树部分分作战表	33
V0: 读入完整 + 合法兜底	34
V1: 特殊性质特判	35
V2: 小图精确	35
V3: 枚举点集/边集	36
V4: 树题暴力	36
V5: 升级路线表	37
GRAPH-12 0-1 BFS 与分层图最短路	38
GRAPH-13 无向图桥与割点	40
TREE-00 二叉树基础	43
TREE-01: 普通树 RootedTree 信息层	48
TREE-02 树直径、换根 DP 与树上差分	52
DP-14: 树形 DP	56
DP-15: DAG DP	59

第 5 卷 图论与树论

考试速查定位

第 5 卷 图论与树论

- 这是第几卷：05。
- 主要查什么：主查图树模板：统一 1-index 建图、无权 BFS、非负权 Dijkstra、小图 Floyd、拓扑、最小生成树、强连通、LCA 和树上 DP。
- 什么时候翻：图/网格/树题，最短路、连通性、拓扑、LCA、树 DP 时翻。
- 题面关键词：BFS/DFS; Dijkstra; Floyd; Bellman-Ford; DSU; MST; Topo; SCC; LCA; 树形 DP。

自动由 03_modules/GRAPH-*.md、TREE-*.md、DP-14/15 重建。定位是统一建图、图论算法、树信息层、树形 DP 与部分分路线。

使用顺序

1. 先看 GRAPH-00 和 OPS-00 的建图协议，确认有向/无向和边权。
2. 普通图问题查 GRAPH-01 到 GRAPH-08。
3. 树题先用 TREE-01 整理 parent/depth/DFS 序，再接 GRAPH-09 或 DP-14。
4. 不会正解时先看 GRAPH-11 的 V0 到 V5 部分分路线。

GRAPH-00 标准 Graph 与建图场景

模块编号：GRAPH-00

模块名称：标准 Graph、建图场景、有向/无向图

标签：[图论][标准容器][建图][1-index]

一句话用途：把题面里的边统一整理成 Graph，后续 BFS、DFS、Dijkstra、Topo、Kruskal、Floyd、Bellman-Ford、SCC、LCA 都尽量共用它。

题面触发词：

- 有 n 个点、 m 条边。
- 道路、航线、关系、依赖、连接、能到达。
- 树、森林、无向图、有向图、带权图。
- 点编号通常是 $1..n$ 。

什么时候用：

- 普通图论题先用本模块建图。
- 需要从点出发遍历时使用 $G.g[u]$ 。
- 需要处理全部边、排序边、初始化矩阵时使用 $G.edges$ 。
- 权值、距离、答案统一优先用 ll 。

不要什么时候用：

- 最大流、最小费用流不要直接用本 Graph，它们需要残量边，见 GRAPH-10。
- 极限内存卡得很死时，可能要链式前向星。
- 网格 BFS 通常直接用二维数组和方向数组，除非题目明显要转图。

复杂度：

- 建图： $O(n + m)$ 空间， $O(m)$ 时间。
- 无向边在 $G.g$ 中存两次，在 $G.edges$ 中只存一次逻辑边。

数据范围参考：

- $n, m \leq 2e5$ 常规可用。
- m 特别大时注意 $\text{vector}<\text{vector}<\text{AdjEdge}>>$ 的常数和内存。

依赖的标准容器：

- Graph。
- 1-index 点编号。
- ll 边权。

输入如何整理：

```

int n, m;
cin >> n >> m;
Graph G(n);
for (int i = 1; i <= m; i++) {
    int u, v;
    ll w;
    cin >> u >> v >> w;
    G.add_undirected(u, v, w); // 无向带权
}

```

接口:

```

Graph G(n)
G.init(n)
G.add_undirected(u, v, w)
G.add_directed(u, v, w)
G.g[u]      从 u 出发的邻接边
G.edges     每条输入逻辑边

```

输出能力:

- 统一邻接表。
- 统一全边表。
- 记录有向/无向信息, 方便 Floyd、Bellman-Ford 等模块。

下游可接:

- DFS/BFS、无权最短路、Dijkstra、Topo、DAG DP、SCC、LCA。
- Floyd、Bellman-Ford、Kruskal。

可拼接模块:

- GRAPH-01 连通性。
- GRAPH-02/03/04 最短路。
- GRAPH-05 Topo/DAG DP。
- GRAPH-06 DSU/Kruskal。
- GRAPH-08 SCC。
- GRAPH-09 树和 LCA。

模板代码:

```

struct AdjEdge {
    int to;
    ll w;
    int edge_index; // 内部边下标, 只给模板内部跳父边/查原边用
    bool directed;
    int input_id;   // 对外边号, 默认 1-index
};

struct FullEdge {
    int from, to;
    ll w;
    bool directed;
    int input_id;   // 对外边号, 默认 1-index
};

struct Graph {
    int n;
    vector<vector<AdjEdge>> g;
    vector<FullEdge> edges;

    Graph(int n = 0) {
        init(n);
    }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
    }
}

```

```

        edges.clear();
    }

    void add_edge_raw(int u, int v, ll w, bool directed) {
        int edge_index = (int)edges.size(); // 内部 0-based, 不输出
        int input_id = edge_index + 1;      // 题面边号统一 1-based
        edges.push_back({u, v, w, directed, input_id});
        g[u].push_back({v, w, edge_index, directed, input_id});
        if (!directed) {
            g[v].push_back({u, w, edge_index, directed, input_id});
        }
    }

    void add_undirected(int u, int v, ll w = 1) {
        add_edge_raw(u, v, w, false);
    }

    void add_directed(int u, int v, ll w = 1) {
        add_edge_raw(u, v, w, true);
    }
};

```

调用示例:

```

Graph G(n);
G.add_undirected(1, 2); // 无向无权
G.add_undirected(2, 3, 5); // 无向带权
G.add_directed(3, 4, 7); // 有向带权 3 -> 4
G.add_directed(4, 5); // 有向无权 4 -> 5, 更不容易写错

for (auto e : G.g[2]) {
    int v = e.to;
    ll w = e.w;
    int input_id = e.input_id; // 1-index, 可直接输出题面边号
}

for (auto e : G.edges) {
    int u = e.from;
    int v = e.to;
}

```

常见坑:

- 无向图只写 add_undirected, 有向图只写 add_directed; 考场不要直接调用 add_edge_raw。
- 不要写 G.add_edge(u, v, true) 这种四参/布尔接口; 本资料主模板故意不暴露它, 避免把 true 当权值。
- Kruskal 只能遍历 G.edges, 不要遍历 G.g, 否则无向边会重复。
- Topo 和 SCC 通常要求有向边, 建图时统一写 G.add_directed(u, v, w); 不要临时改回布尔参数接口。
- Floyd 初始化时要看 e.directed, 无向边要对称赋值。
- 多组数据必须重新 G.init(n)。
- 如果题目点编号是 0..n-1, 读入时立刻 u++, v++ 转成内部 1..n, 后续不要切换到 0-index。
- 点编号默认 1-index; 对外边号用 input_id, 也是 1-index。内部 edge_index 不输出, 只给 Lowlink 跳父边等模板内部使用。

图类型与模块路由协议

图类型	建图方式	优先模块	误用风险
无向无权图	add_undirected(u,v)	DFS/BFS、连通块、二分图	Topo/SCC 不适用
无向带权图	add_undirected(u,v,w)	Dijkstra、Kruskal、树 LCA	Kruskal 遍历 edges, 不要遍历 g
有向无权图	add_directed(u,v)	BFS、Topo、SCC	不要临时改回布尔参数接口
有向带权图	add_directed(u,v,w)	Dijkstra、Bellman-Ford、DAG DP	有负边不能 Dijkstra
DAG	全部有向边且无环	Topo、DAG DP	topo.size()!=n 说明有环
树	无向、连通、m=n-1	TreeDFS、LCA、树形 DP	一般图不能当树
流网络	FlowGraph	Dinic	不用普通 Graph

暴力/部分替代：

- 点很少时可以用邻接矩阵 `dist[n+1][n+1]`。
- 只判断一两次连通时，可临时用二维 `bool adj` + DFS。

升级方向：

- 普通无权图：接 BFS。
- 非负权：接 Dijkstra。
- 小图全源：接 Floyd。
- 最小生成树：接 DSU/Kruskal。
- 树路径：接 DFS distance + LCA。

最小测试样例：

```
n=3
add 1 2 undirected
add 2 3 directed
G.g[1] 有 2
G.g[2] 有 1 和 3
G.g[3] 没有 2
G.edges.size() = 2
```

GRAPH-01 DFS/BFS 连通与遍历

模块编号：GRAPH-01

模块名称：DFS/BFS 连通、连通块、可达性

标签：[图论][DFS][BFS][连通性][可达性]

一句话用途：从某个点出发访问所有能到达的点，或把整张图分成若干连通块。

题面触发词：

- 是否连通、能不能到达。
- 有几个连通块。
- 从 s 出发能访问哪些点。
- 岛屿、网络、关系传递。

什么时候用：

- 无权图只关心可达，不关心最短距离。
- 无向图统计连通块。
- 有向图从一个起点统计可达点。
- 树上从根遍历父子关系。

不要什么时候用：

- 要最短路距离时优先用 GRAPH-02 无权 BFS 或 GRAPH-03 Dijkstra。
- 有很多次动态加边连通查询时，优先 DSU。
- 有向图要求强连通分量时，用 SCC。
- 递归深度可能到 $2e5$ 时，DFS 递归可能爆栈，优先 BFS 或手写栈。

复杂度：

- $O(n + m)$ 时间。
- $O(n)$ 额外空间。

数据范围参考：

- $n, m \leq 2e5$ 常规可用。
- 深链树建议 BFS 或迭代 DFS。

依赖的标准容器：

- Graph。
- 遍历从 `G.g[u]` 出发。

输入如何整理：

```

Graph G(n);
for (int i = 1; i <= m; i++) {
    int u, v;
    cin >> u >> v;
    G.add_undirected(u, v); // 无向连通性
}

```

接口:

```

vector<int> bfs_order(const Graph& G, int s)
vector<int> dfs_order_iterative(const Graph& G, int s)
vector<int> connected_component_id_undirected(const Graph& G, int& cnt)

```

输出能力:

- 一次遍历顺序。
- 每个点所属连通块编号。
- 连通块数量。

下游可接:

- 连通块内分别做 DP。
- 检查图是否连通后再跑 Kruskal、树算法。
- BFS 最短路模块。

可拼接模块:

- Graph + DFS/BFS + TreeDFS。
- Graph + BFS + BipartiteColor。
- Graph + DSU 连通性互相替换。

模板代码:

```

vector<int> bfs_order(const Graph &G, int s) {
    vector<int> vis(G.n + 1, 0), order;
    queue<int> q;
    vis[s] = 1;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for (auto e : G.g[u]) {
            int v = e.to;
            if (vis[v]) continue;
            vis[v] = 1;
            q.push(v);
        }
    }
    return order;
}

vector<int> dfs_order_iterative(const Graph &G, int s) {
    vector<int> vis(G.n + 1, 0), order;
    vector<int> st;
    st.push_back(s);
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        if (vis[u]) continue;
        vis[u] = 1;
        order.push_back(u);
        for (auto e : G.g[u]) {
            int v = e.to;
            if (!vis[v]) st.push_back(v);
        }
    }
    return order;
}

```

```
}
```

// 仅限无向图连通块; 有向图强连通请用 SCC。

```
vector<int> connected_component_id_undirected(const Graph &G, int &cnt) {  
    vector<int> comp(G.n + 1, 0);  
    cnt = 0;  
    for (int s = 1; s <= G.n; s++) {  
        if (comp[s]) continue;  
        cnt++;  
        queue<int> q;  
        comp[s] = cnt;  
        q.push(s);  
        while (!q.empty()) {  
            int u = q.front();  
            q.pop();  
            for (auto e : G.g[u]) {  
                int v = e.to;  
                if (comp[v]) continue;  
                comp[v] = cnt;  
                q.push(v);  
            }  
        }  
    }  
    return comp;  
}
```

调用示例:

```
auto order = bfs_order(G, 1);  
  
int cnt = 0;  
auto comp = connected_component_id_undirected(G, cnt);  
cout << cnt << "\n";  
cout << (comp[x] == comp[y] ? "YES" : "NO") << "\n";
```

常见坑:

- 有向图的可达性不是无向连通性, `G.add_directed(u,v)` 只能从 `u` 到 `v`。
- 递归 DFS 在长链上容易爆栈。
- 连通块编号要从所有未访问点启动, 而不是只从 1。
- 多组数据 `vis/comp` 要重新开。
- 自环不会影响连通性, 重边只会多遍历几次。

暴力/部分替代:

- `n <= 500` 可用邻接矩阵 + DFS。
- 查询次数少时, 每次从起点 BFS 判断可达。
- 静态多次连通查询可升级为一次连通块编号或 DSU。

升级方向:

- 可达 + 最短步数: 换无权 BFS。
- 多次合并集合: 换 DSU。
- 有向强连通: 换 SCC。
- 树上遍历: 接树上 DFS 距离。

最小测试样例:

```
4 2  
1 2  
3 4  
连通块数 = 2  
comp[1] == comp[2]  
comp[1] != comp[3]
```

GRAPH-02 无权最短路 BFS

模块编号: GRAPH-02

模块名称: 无权图最短路 BFS

标签: [图论][BFS][无权最短路][最少步数]

一句话用途: 在每条边代价都一样时, 用 BFS 求从起点到所有点的最少边数。

题面触发词:

- 最少经过几条边。
- 最少操作次数。
- 每次移动代价相同。
- 无权图、边权全为 1。

什么时候用:

- 所有边权都是 1。
- 网格每步上下左右代价相同。
- 状态图每次操作代价相同。
- 需要最短步数和父节点恢复路径。

不要什么时候用:

- 边权不是全相等时不要用普通 BFS。
- 边权非负但有不同值, 用 Dijkstra。
- 有负权边, 用 Bellman-Ford/SPFA。
- 只判断连通性, 不要多维护距离也可以。

复杂度:

- $O(n + m)$ 时间。
- $O(n)$ 空间。

数据范围参考:

- $n, m \leq 2e5$ 或更大都可。
- 网格 BFS 复杂度约 $O(n*m)$ 。

依赖的标准容器:

- Graph。
- 从 $G.g[u]$ 遍历邻接点。

输入如何整理:

```
Graph G(n);
for (int i = 1; i <= m; i++) {
    int u, v;
    cin >> u >> v;
    G.add_undirected(u, v); // 无向无权
}
```

接口:

```
vector<int> bfs_unweighted(const Graph& G, int s)
vector<int> multi_source_bfs(const Graph& G, const vector<int>& sources)
BfsResult bfs_with_parent(const Graph& G, int s)
```

输出能力:

- $dist[v]$: 从 s 到 v 的最少边数, 不可达为 -1。
- $pre[v]$: 最短路树中的前驱, 可接路径恢复。

下游可接:

- 路径恢复。
- 分层图 DP。
- 二分图染色。
- 多源 BFS。

可拼接模块:

- Graph + BFS + restore_path。

- Graph + BFS + MultiSource.
- Grid + BFS.

模板代码:

```
struct BfsResult {
    vector<int> dist;
    vector<int> pre;
};

vector<int> bfs_unweighted(const Graph &G, int s) {
    vector<int> dist(G.n + 1, -1);
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (auto e : G.g[u]) {
            int v = e.to;
            if (dist[v] != -1) continue;
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
    return dist;
}

BfsResult bfs_with_parent(const Graph &G, int s) {
    BfsResult res;
    res.dist.assign(G.n + 1, -1);
    res.pre.assign(G.n + 1, 0);
    queue<int> q;
    res.dist[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (auto e : G.g[u]) {
            int v = e.to;
            if (res.dist[v] != -1) continue;
            res.dist[v] = res.dist[u] + 1;
            res.pre[v] = u;
            q.push(v);
        }
    }
    return res;
}

vector<int> multi_source_bfs(const Graph &G, const vector<int> &sources) {
    vector<int> dist(G.n + 1, -1);
    queue<int> q;
    for (int s : sources) {
        if (s < 1 || s > G.n) continue;
        if (dist[s] != -1) continue;
        dist[s] = 0;
        q.push(s);
    }
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (auto e : G.g[u]) {
            int v = e.to;
            if (dist[v] != -1) continue;
        }
    }
}
```

```

        dist[v] = dist[u] + 1;
        q.push(v);
    }
}
return dist;
}

```

调用示例:

```

auto res = bfs_with_parent(G, s);
cout << res.dist[t] << "\n";

vector<int> sources = {1, 5, 9};
auto dist_nearest_source = multi_source_bfs(G, sources);

```

常见坑:

- BFS 只适合边权相同的最短路。
- 不可达点 $\text{dist}=-1$, 输出前按题意处理。
- 有向无权图用 `G.add_directed(...)` 建边; 无向无权图用 `G.add_undirected(...)`。
- 如果要路径, 第一次到达时记录 $\text{pre}[v]=u$ 。
- 网格 BFS 不一定要转 Graph, 直接二维数组更短。

暴力/部分替代:

- $n \leq 20$ 可以 DFS 枚举简单路径取最短, 但容易指数爆炸。
- $n \leq 500$ 可用 Floyd 求全源, 但无权单源 BFS 更优。

升级方向:

- 边权变成非负数: Dijkstra。
- 多个起点: 多源 BFS。
- 需要输出路径: 接 `restore_path`。

最小测试样例:

```

4 3
1 2
2 3
1 4
s=1
dist[1]=0 dist[2]=1 dist[3]=2 dist[4]=1

```

GRAPH-03 Dijkstra、路径恢复、多源最短路

模块编号: GRAPH-03

模块名称: Dijkstra、路径恢复、多源最短路

标签: [图论][Dijkstra][非负权][路径恢复][多源最短路]

一句话用途: 边权非负时求最短路, 并可记录前驱恢复一条最短路径; 多个起点时把所有源点一起入堆。

题面触发词:

- 边权非负、道路长度、花费、时间。
- 从一个点到其他点的最短距离。
- 从多个仓库/医院/起点出发到最近目标。
- 输出最短路径经过的点。

什么时候用:

- 边权 $w \geq 0$ 。
- n, m 大, 不能 Floyd。
- 单源或多源最短路。
- 需要恢复一条最短路径。

不要什么时候用:

- 有负权边, 不能用 Dijkstra。

- $n \leq 500$ 且要求任意两点最短路, 可用 Floyd。
- 边权全为 1 时 BFS 更简单。
- 要第 k 短路、动态最短路时, 本模板不够。

复杂度:

- $O((n + m) \log n)$ 时间。
- $O(n + m)$ 空间。

数据范围参考:

- $n, m \leq 2e5$ 常规可用。
- 距离可能很大, 用 `ll`。

依赖的标准容器:

- `Graph`。
- 从 `G.g[u]` 遍历出边。
- `priority_queue`。
- 依赖主骨架: `using ll = long long; const ll LINF = 4'000'000'000'000'000'000LL;`

输入如何整理:

```
Graph G(n);
for (int i = 1; i <= m; i++) {
    int u, v;
    ll w;
    cin >> u >> v >> w;
    G.add_undirected(u, v, w); // 有向边改用 G.add_directed(u, v, w)
}
```

接口:

```
vector<ll> dijkstra(const Graph& G, int s)
ShortestPathResult dijkstra_with_parent(const Graph& G, int s)
ShortestPathResult dijkstra_multi_source(const Graph& G, vector<int> sources)
vector<int> restore_path(pre, s, t)
```

输出能力:

- `dist[v]`: 最短距离, 不可达为 `LINF`。
- `pre[v]`: 恢复路径用的前驱点。
- 多源版本中 `pre[source]=0`。

下游可接:

- 路径恢复。
- 最短路 DAG。
- 二分答案检查。
- 树上路径特判。

可拼接模块:

- `Graph + Dijkstra + restore_path`。
- `Graph + MultiSourceDijkstra`。
- `Dijkstra + DP`。

模板代码:

```
struct ShortestPathResult {
    vector<ll> dist;
    vector<int> pre;
};

ShortestPathResult dijkstra_with_parent(const Graph &G, int s) {
    ShortestPathResult res;
    res.dist.assign(G.n + 1, LINF);
    res.pre.assign(G.n + 1, 0);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
    res.dist[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
```

```

        auto [du, u] = pq.top();
        pq.pop();
        if (du != res.dist[u]) continue;
        for (auto e : G.g[u]) {
            int v = e.to;
            ll w = e.w;
            if (res.dist[u] + w < res.dist[v]) {
                res.dist[v] = res.dist[u] + w;
                res.pre[v] = u;
                pq.push({res.dist[v], v});
            }
        }
    }
    return res;
}

vector<ll> dijkstra(const Graph &G, int s) {
    return dijkstra_with_parent(G, s).dist;
}

ShortestPathResult dijkstra_multi_source(const Graph &G, const vector<int> &sources) {
    ShortestPathResult res;
    res.dist.assign(G.n + 1, LINF);
    res.pre.assign(G.n + 1, 0);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
    for (int s : sources) {
        if (s < 1 || s > G.n) continue;
        if (res.dist[s] == 0) continue;
        res.dist[s] = 0;
        pq.push({0, s});
    }
    while (!pq.empty()) {
        auto [du, u] = pq.top();
        pq.pop();
        if (du != res.dist[u]) continue;
        for (auto e : G.g[u]) {
            int v = e.to;
            ll w = e.w;
            if (res.dist[u] + w < res.dist[v]) {
                res.dist[v] = res.dist[u] + w;
                res.pre[v] = u;
                pq.push({res.dist[v], v});
            }
        }
    }
    return res;
}

vector<int> restore_path(const vector<int> &pre, int s, int t) {
    vector<int> path;
    for (int cur = t; cur != 0; cur = pre[cur]) {
        path.push_back(cur);
        if (cur == s) break;
    }
    if (path.empty() || path.back() != s) return {};
    reverse(path.begin(), path.end());
    return path;
}

```

调用示例:

```

auto res = dijkstra_with_parent(G, s);
if (res.dist[t] == LINF) {
    cout << "-1\n";
}

```

```

} else {
    cout << res.dist[t] << "\n";
    auto path = restore_path(res.pre, s, t);
}

vector<int> sources = {1, 5, 9};
auto multi = dijkstra_multi_source(G, sources);

```

常见坑:

- 有负权边时会错。
- $LINF + w$ 可能溢出, 只有从堆中弹出的可达点才松弛。
- 堆里旧状态要用 $du \neq dist[u]$ 跳过。
- 无向图用 `G.add_undirected(...)`; 有向图用 `G.add_directed(...)`。
- 多源最短路不是跑多次 Dijkstra, 而是所有源点距离设 0 后一起入堆。
- `restore_path` 只能恢复到起点可达的目标。

暴力/部分替代:

- $n \leq 500$ 可用 Floyd。
- m 很小、点很少可用 Bellman-Ford。
- 边权全为 1 时用 BFS。

升级方向:

- 需要多次任意两点查询: 小图 Floyd; 大图通常要换模型。
- 需要方案数: 在 Dijkstra 松弛时维护 `cnt[v]`。
- 需要路径限制: 可能变成状态最短路, 把状态拆成点。

最小测试样例:

```

4 4
1 2 5
1 3 1
3 2 1
2 4 2
s=1
dist[4]=4
path: 1 3 2 4

```

GRAPH-04 Floyd、Bellman-Ford 与 SPFA

模块编号: GRAPH-04

模块名称: Floyd、Bellman-Ford、SPFA

标签: [图论][全源最短路][负权边][负环][Floyd][Bellman-Ford][SPFA]

一句话用途: 小图用 Floyd 求任意两点最短路; 有负权边时用 Bellman-Ford 或 SPFA, 并能判断负环风险。

题面触发词:

- 任意两点最短路、多次问 u 到 v 。
- $n \leq 300/500$ 。
- 边权可能为负。
- 判断是否存在负环。

什么时候用:

- Floyd: 点数小, 需要全源最短路。
- Bellman-Ford: 有负权边, 单源最短路, 且想稳妥判断负环。
- SPFA: 有负权边且图比较稀疏时可尝试, 但不能保证最坏复杂度。

不要什么时候用:

- n 很大时不要 Floyd。
- 非负权大图优先 Dijkstra。
- 边权全为 1 优先 BFS。
- 有负环且题目要求最短距离时, 要先判断“最短路是否存在”。

复杂度:

- Floyd: $O(n^3)$ 时间, $O(n^2)$ 空间。
- Bellman-Ford: $O(nm)$ 时间, $O(n)$ 空间。
- SPFA: 平均可能较快, 最坏 $O(nm)$ 。

数据范围参考:

- Floyd: $n \leq 300/500$ 。
- Bellman-Ford: $n*m$ 能过时使用。
- SPFA: 竞赛中要谨慎, 能 Dijkstra 就别 SPFA。

依赖的标准容器:

- Graph。
- Floyd 和 Bellman-Ford 主要遍历 `G.edges`。
- SPFA 遍历 `G.g[u]`。
- 依赖主骨架: `using ll = long long; const ll LINF = 4'000'000'000'000'000'000LL;`

输入如何整理:

```
Graph G(n);
for (int i = 1; i <= m; i++) {
    int u, v;
    ll w;
    cin >> u >> v >> w;
    G.add_directed(u, v, w); // 无向边改用 G.add_undirected(u, v, w)
}
```

接口:

```
void floyd(const Graph& G)
全局 floyd_dist[u][v]
BellmanResult bellman_ford(const Graph& G, int s)
BellmanResult spfa(const Graph& G, int s)
```

输出能力:

- Floyd: `dist[u][v]` 任意两点最短路。
- Bellman-Ford/SPFA: `dist[v]`, 以及是否检测到从源点可达的负环。

下游可接:

- 全源距离矩阵可接状压 DP。
- 负环判断可接可行性输出。
- 单源负权最短路可接路径类题。

可拼接模块:

- Graph.edges + Floyd。
- Graph.edges + Bellman-Ford。
- Graph.g + SPFA。
- Floyd + BitmaskDP。

模板代码:

```
struct BellmanResult {
    vector<ll> dist;
    bool has_negative_cycle = false;
};

const int MAXF = 505; // Floyd 一般只给小图用, 按题目 n 上限改
static ll floyd_dist[MAXF][MAXF];

void floyd(const Graph &G) {
    assert(G.n > 0 && G.n < MAXF);
    for (int i = 1; i <= G.n; i++) {
        for (int j = 1; j <= G.n; j++) {
            floyd_dist[i][j] = (i == j ? 0 : LINF);
        }
    }
}
```

```

    for (auto e : G.edges) {
        floyd_dist[e.from][e.to] = min(floyd_dist[e.from][e.to], e.w);
        if (!e.directed) floyd_dist[e.to][e.from] = min(floyd_dist[e.to][e.from], e.w);
    }
    for (int k = 1; k <= G.n; k++) {
        for (int i = 1; i <= G.n; i++) {
            if (floyd_dist[i][k] == LINF) continue;
            for (int j = 1; j <= G.n; j++) {
                if (floyd_dist[k][j] == LINF) continue;
                floyd_dist[i][j] = min(floyd_dist[i][j], floyd_dist[i][k] + floyd_dist[k][j]);
            }
        }
    }
}

BellmanResult bellman_ford(const Graph &G, int s) {
    BellmanResult res;
    res.dist.assign(G.n + 1, LINF);
    res.dist[s] = 0;
    for (int i = 1; i <= G.n - 1; i++) {
        bool changed = false;
        for (auto e : G.edges) {
            if (res.dist[e.from] != LINF && res.dist[e.from] + e.w < res.dist[e.to]) {
                res.dist[e.to] = res.dist[e.from] + e.w;
                changed = true;
            }
            if (!e.directed && res.dist[e.to] != LINF && res.dist[e.to] + e.w < res.dist[e.from]) {
                res.dist[e.from] = res.dist[e.to] + e.w;
                changed = true;
            }
        }
        if (!changed) break;
    }
    for (auto e : G.edges) {
        if (res.dist[e.from] != LINF && res.dist[e.from] + e.w < res.dist[e.to]) {
            res.has_negative_cycle = true;
        }
        if (!e.directed && res.dist[e.to] != LINF && res.dist[e.to] + e.w < res.dist[e.from]) {
            res.has_negative_cycle = true;
        }
    }
    return res;
}

BellmanResult spfa(const Graph &G, int s) {
    BellmanResult res;
    res.dist.assign(G.n + 1, LINF);
    vector<int> inq(G.n + 1, 0), cnt(G.n + 1, 0);
    queue<int> q;
    res.dist[s] = 0;
    q.push(s);
    inq[s] = 1;
    cnt[s] = 1;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = 0;
        for (auto e : G.g[u]) {
            int v = e.to;
            ll w = e.w;
            if (res.dist[u] + w >= res.dist[v]) continue;
            res.dist[v] = res.dist[u] + w;
            if (!inq[v]) {

```

```

        q.push(v);
        inq[v] = 1;
        cnt[v]++;
        if (cnt[v] > G.n) {
            res.has_negative_cycle = true;
            return res;
        }
    }
}
return res;
}
}
}

```

调用示例:

```

floyd(G);
cout << (floyd_dist[u][v] == LINF ? -1 : floyd_dist[u][v]) << "\n";

```

```

auto res = bellman_ford(G, s);
if (res.has_negative_cycle) cout << "NEGATIVE CYCLE\n";

```

常见坑:

- Floyd 初始化要处理重边取最小。
- Floyd 中 LINF 不能直接相加, 先判断可达。
- Bellman-Ford 无向负边等价于负环风险, 题目一般不会这样给最短路正解。
- Bellman-Ford 对无向边要松弛两个方向。
- SPFA 可能被卡, 不是万能加速版。
- 负环判断通常只判断“从源点可达”的负环。

暴力/部分替代:

- 小图任意两点可 Floyd。
- 没负边时用 Dijkstra。
- $n \leq 50$ 可直接矩阵松弛。

升级方向:

- 大图非负权: 换 Dijkstra。
- 任意两点 + 必经若干点: Floyd 后接状压 DP。
- 需要路径恢复: Floyd 维护 nxt 或 Bellman-Ford 维护 pre。

最小测试样例:

```

3 3
1 2 4
1 3 10
2 3 -2
Bellman-Ford from 1: dist[3]=2
Floyd: dist[1][3]=2

```

GRAPH-05 拓扑排序与 DAG DP

模块编号: GRAPH-05

模块名称: Topo、DAG DP

标签: [图论][拓扑排序][DAG][依赖顺序][DP]

一句话用途: 有向无环图中先求合法顺序, 再按拓扑序做路径计数、最长路、最短路或依赖 DP。

题面触发词:

- 依赖、先修、前置任务。
- 有向无环图、DAG。
- 必须先完成 A 才能完成 B。
- 问方案数、最长链、最早完成时间。

什么时候用:

- 图是有向图，且没有环。
- 转移依赖方向明确。
- DP 状态沿边传播，要求先算前驱再算后继。
- 需要判断是否存在环。

不要什么时候用：

- 无向图不能直接拓扑排序。
- 有向图有环时，普通 DAG DP 不成立。
- 边权有正负但要求一般最短路时，不要硬套 DAG DP，除非图确实无环。
- 树上 DP 用树 DFS 更直接。

复杂度：

- 拓扑排序： $O(n + m)$ 。
- DAG DP：通常 $O(n + m)$ 。

数据范围参考：

- $n, m \leq 2e5$ 常规可用。
- 方案数可能很大，按题目取模。

依赖的标准容器：

- Graph。
- Topo 只适用于 `G.add_directed(u, v)` 建出的有向边；无向边不能直接参与拓扑排序。
- Topo 入度建议从 `G.edges` 统计有向边。
- DP 转移从 `G.g[u]` 出发，并跳过非有向边。
- 依赖主骨架：using `ll = long long`; const `ll LINF = 4'000'000'000'000'000'000LL`;

输入如何整理：

```
Graph G(n);
for (int i = 1; i <= m; i++) {
    int u, v;
    cin >> u >> v;
    G.add_directed(u, v); // u 必须在 v 前
}
```

接口：

```
vector<int> topo_sort(const Graph& G)
vector<ll> dag_count_paths(const Graph& G, int s, ll mod)
vector<ll> dag_longest_path(const Graph& G, int s)
```

输出能力：

- 一个拓扑序。
- 如果拓扑序长度小于 n ，说明有环。
- DAG 上从源点出发的路径方案数或最长路。

下游可接：

- DAG DP。
- 任务排程。
- 最短路 DAG 计数。
- 依赖合法性判断。

可拼接模块：

- Graph + Topo + DP。
- Dijkstra + ShortestPathDAG + Topo。
- Topo + Queue。

模板代码：

```
vector<int> topo_sort(const Graph &G) {
    for (auto e : G.edges) {
        if (!e.directed) return {};
    }
    vector<int> indeg(G.n + 1, 0);
    for (auto e : G.edges) {
        if (!e.directed) continue;
    }
}
```

```

        indeg[e.to]++;
    }
    queue<int> q;
    for (int i = 1; i <= G.n; i++) {
        if (indeg[i] == 0) q.push(i);
    }
    vector<int> order;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for (auto e : G.g[u]) {
            if (!e.directed) continue;
            int v = e.to;
            indeg[v]--;
            if (indeg[v] == 0) q.push(v);
        }
    }
    return order;
}

vector<ll> dag_count_paths(const Graph &G, int s, ll mod) {
    auto order = topo_sort(G);
    if ((int)order.size() != G.n) return {};
    vector<ll> dp(G.n + 1, 0);
    dp[s] = 1;
    for (int u : order) {
        for (auto e : G.g[u]) {
            if (!e.directed) continue;
            int v = e.to;
            dp[v] = (dp[v] + dp[u]) % mod;
        }
    }
    return dp;
}

vector<ll> dag_longest_path(const Graph &G, int s) {
    auto order = topo_sort(G);
    if ((int)order.size() != G.n) return {};
    vector<ll> dp(G.n + 1, -LINF);
    dp[s] = 0;
    for (int u : order) {
        if (dp[u] == -LINF) continue;
        for (auto e : G.g[u]) {
            if (!e.directed) continue;
            int v = e.to;
            dp[v] = max(dp[v], dp[u] + e.w);
        }
    }
    return dp;
}

```

调用示例:

```

auto order = topo_sort(G);
if ((int)order.size() < n) {
    cout << "cycle\n";
} else {
    auto ways = dag_count_paths(G, 1, 1000000007LL);
    cout << ways[n] << "\n";
}

```

常见坑:

- Topo 只对有向图有意义, 标准 Graph 中必须用 `G.add_directed(u, v, w)` 建边。

- 建边方向要按“前置->后继”，不要反。
- `order.size() < n` 表示有环，DAG DP 结果不可用。
- 本页两个 DAG DP 函数遇到有环会返回空数组，调用处要判断 `empty()`。
- 无向边放进 Topo 会让入度统计失真。
- 多源 DAG DP 可以把所有源点 `dp[s]=1`。
- 最长路初始化要用 `-INF`，不可达不能参与转移。

暴力/部分替代：

- $n \leq 20$ 可 DFS 枚举所有路径。
- 无权最长链小数据可记忆化 DFS。
- 只判断环，小数据可 DFS 三色标记。

升级方向：

- DFS 记忆化 DAG DP -> Topo 表推。
- 依赖顺序 + 资源限制可能升级为普通 DP/贪心。
- 最短路 DAG 计数：先 Dijkstra，再只保留满足 $\text{dist}[u] + w = \text{dist}[v]$ 的边做 DP。

最小测试样例：

```
4 4
1 2
1 3
2 4
3 4
Topo 可能为 1 2 3 4
从 1 到 4 路径数 = 2
```

GRAPH-06 DSU 与 Kruskal 最小生成树

模块编号：GRAPH-06

模块名称：DSU、Kruskal、最小生成树

拼接提醒：本模块和 DS-04 都给了 DSU。写 MST 时优先保留本模块的 `struct DSU`，如果前面已经复制过 DSU，就不要再复制第二份。

标签：[\[图论\]](#)[\[并查集\]](#)[\[Kruskal\]](#)[\[最小生成树\]](#)

一句话用途：无向带权图中选出连接所有点且总权值最小的 $n-1$ 条边。

题面触发词：

- 最小生成树、修路最小成本。
- 连接所有点、总代价最小。
- 无向带权图。
- 判断能不能把所有点连通。

什么时候用：

- 图是无向图。
- 要连通所有点，边权总和最小。
- 只需要一棵生成树，不需要起点到终点最短路。
- 边可以排序。

不要什么时候用：

- 有向图最小树形图不是 Kruskal。
- 问两点最短路径不要用 MST 代替。
- 图不连通时没有生成树，只能得到森林。
- 动态加删边 MST 不是本模板。

复杂度：

- 排序 $O(m \log m)$ 。
- DSU 合并近似 $O(1)$ 均摊。

数据范围参考：

- $n, m \leq 2e5$ 常规可用。
- 权值和用 `ll`。

依赖的标准容器:

- Graph。
- 只遍历 G.edges。
- DSU。

输入如何整理:

```
Graph G(n);
for (int i = 1; i <= m; i++) {
    int u, v;
    ll w;
    cin >> u >> v >> w;
    G.add_undirected(u, v, w); // Kruskal 必须按无向边理解
}
```

接口:

DSU.init(n), find(x), unite(a,b), same(a,b)

KruskalResult kruskal(const Graph& G)

输出能力:

- MST 总权值。
- 是否成功连接所有点。
- 被选中的边编号。

下游可接:

- 树上 DFS/LCA。
- 最大边最小化瓶颈树。
- 连通性检查。

可拼接模块:

- Graph.edges + DSU + Kruskal。
- Kruskal + TreeDFS。
- Kruskal + LCA。

模板代码:

```
struct DSU {
    vector<int> fa, sz;

    DSU(int n = 0) {
        if (n > 0) init(n);
    }

    void init(int n) {
        fa.resize(n + 1);
        sz.assign(n + 1, 1);
        iota(fa.begin(), fa.end(), 0);
    }

    int find(int x) {
        while (x != fa[x]) {
            fa[x] = fa[fa[x]];
            x = fa[x];
        }
        return x;
    }

    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        fa[b] = a;
        sz[a] += sz[b];
    }
};
```

```

        return true;
    }

    bool same(int a, int b) {
        return find(a) == find(b);
    }
};

struct KruskalResult {
    ll total = 0;
    bool connected = false;
    vector<int> chosen_input_ids; // 1-index 题面边号
};

KruskalResult kruskal(const Graph &G) {
    vector<int> ord((int)G.edges.size());
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(), ord.end(), [&](int a, int b) {
        return G.edges[a].w < G.edges[b].w;
    });

    DSU dsu;
    dsu.init(G.n);
    KruskalResult res;
    for (int id : ord) {
        auto e = G.edges[id];
        if (e.directed) continue; // MST 只处理无向边
        if (dsu.unite(e.from, e.to)) {
            res.total += e.w;
            res.chosen_input_ids.push_back(e.input_id);
        }
    }
    res.connected = ((int)res.chosen_input_ids.size() == G.n - 1);
    return res;
}

```

调用示例:

```

auto mst = kruskal(G);
if (!mst.connected) {
    cout << "orz\n";
} else {
    cout << mst.total << "\n";
}

```

常见坑:

- Kruskal 遍历 `G.edges`, 不要遍历 `G.g`。
- MST 是无向图问题, 建边要用 `G.add_undirected(u, v, w)`; 输入有向边不能直接套。
- 图不连通时选不到 $n-1$ 条边。
- 边权可能为负, Kruskal 仍然可用。
- 如果题目要求最大生成树, 把排序改成从大到小。
- total 用 `ll`。

暴力/部分分替代:

- $n \leq 10$ 可枚举边子集, 选 $n-1$ 条判断连通。
- m 小时可用暴力加边排序仍然是 Kruskal 的雏形。
- 只判断连通时直接 DSU, 不用排序。

升级方向:

- MST 建出树后可接 LCA 做树上最大边查询。
- 最小瓶颈生成树直接用 Kruskal 的最大入选边。
- 稠密图也可用 Prim, 但本卷统一优先 Kruskal。

最小测试样例:

```
4 5
1 2 1
2 3 2
3 4 3
1 4 10
1 3 5
MST total = 6
```

GRAPH-07 二分图染色与二分图匹配

模块编号: GRAPH-07

模块名称: 二分图染色、二分图匹配

标签: [图论][二分图][染色][匹配]

一句话用途: 用 BFS/DFS 染色判断图能否分成左右两部; 已知左右部时用匹配求最多配对数。

题面触发词:

- 分成两组, 组内不能有冲突。
- 奇环判断。
- 男生女生、机器任务、左部右部配对。
- 每个对象最多匹配一个。

什么时候用:

- 二分图判断: 无向图, 要求相邻点颜色不同。
- 二分图匹配: 左部点和右部点之间有可选边, 每点最多选一条。
- 数据规模中等, 匈牙利/Kuhn 能过。

不要什么时候用:

- 一般图最大匹配不是这个模板。
- 带权匹配不是这个模板。
- 需要最大流建模也可以用 Dinic, 但本模块更短。
- 图不是天然左右部时, 先染色再决定左右部。

复杂度:

- 染色: $O(n + m)$ 。
- Kuhn 匹配: $O(nL * m)$ 最坏, 常用于中小规模。

数据范围参考:

- 染色可到 $2e5$ 。
- Kuhn 匹配适合 $nL, nR, m \leq 2000 \sim 5000$ 视时限而定。
- 更大二分图匹配建议 Dinic 或 Hopcroft-Karp。

依赖的标准容器:

- Graph。
- 染色遍历 `G.g[u]`。
- 匹配模板独立用 `adjL[left].push_back(right)`, 右部也保持 `1..nR`, 不做偏移。

输入如何整理:

```
// 二分图判断: 无向冲突边
```

```
Graph G(n);
G.add_undirected(u, v);
```

```
// 二分图匹配: 左部 1..nL, 右部 1..nR
```

```
adjL[left].push_back(right);
```

接口:

```
BipartiteColorResult bipartite_color(const Graph& G)
MatchingResult bipartite_matching_kuhn_result(vector<int> adjL[], int nL, int nR)
int bipartite_matching_kuhn(vector<int> adjL[], int nL, int nR)
```

输出能力:

- 是否为二分图。
- 每个点颜色 1/2。
- 最大匹配数。
- 右部每个点匹配到的左部点。

下游可接：

- 分组可行性。
- 最小点覆盖、最大独立集等进阶结论。
- Dinic 最大流替代。

可拼接模块：

- Graph + BFS Color。
- adjL[1..nL] + Kuhn Matching。
- Bipartite + Dinic。

模板代码：

```
struct BipartiteColorResult {
    bool ok = true;
    vector<int> color;
};

BipartiteColorResult bipartite_color(const Graph &G) {
    BipartiteColorResult res;
    res.color.assign(G.n + 1, 0);
    for (int s = 1; s <= G.n; s++) {
        if (res.color[s]) continue;
        queue<int> q;
        res.color[s] = 1;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto e : G.g[u]) {
                int v = e.to;
                if (!res.color[v]) {
                    res.color[v] = 3 - res.color[u];
                    q.push(v);
                } else if (res.color[v] == res.color[u]) {
                    res.ok = false;
                }
            }
        }
    }
    return res;
}

bool kuhn_dfs(int u, vector<int> adjL[], vector<int> &vis, vector<int> &matchR, int tag) {
    vis[u] = tag;
    for (int r : adjL[u]) {
        if (matchR[r] == 0 || (vis[matchR[r]] != tag && kuhn_dfs(matchR[r], adjL, vis, matchR, tag))) {
            matchR[r] = u;
            return true;
        }
    }
    return false;
}

struct MatchingResult {
    int size = 0;
    vector<int> matchR; // matchR[r] = matched Left node, 0 means unmatched
};

MatchingResult bipartite_matching_kuhn_result(vector<int> adjL[], int nL, int nR) {
```

```

vector<int> matchR(nR + 1, 0);
vector<int> vis(nL + 1, 0);
MatchingResult res;
for (int u = 1; u <= nL; u++) {
    if (kuhn_dfs(u, adjL, vis, matchR, u)) res.size++;
}
res.matchR = matchR;
return res;
}

int bipartite_matching_kuhn(vector<int> adjL[], int nL, int nR) {
    return bipartite_matching_kuhn_result(adjL, nL, nR).size;
}

```

调用示例:

```

auto color = bipartite_color(G);
cout << (color.ok ? "YES" : "NO") << "\n";

const int MAXL = 200000 + 5;
static vector<int> adjL[MAXL];
adjL[1].push_back(2);
cout << bipartite_matching_kuhn(adjL, nL, nR) << "\n";

auto mt = bipartite_matching_kuhn_result(adjL, nL, nR);
for (int r = 1; r <= nR; r++) {
    if (mt.matchR[r]) cout << mt.matchR[r] << " - " << r << '\n';
}

```

常见坑:

- 二分图染色针对无向冲突图最常见, 有向边要先理解题意。
- 图不连通也要从每个未染色点启动。
- 自环一定破坏二分图。
- 匹配模板要求左部 $1..nL$ 、右部 $1..nR$; 不要把右部偏移成全局点号。
- Kuhn 的 vis 是每轮 DFS 的访问标记, 不是全局永久访问。
- 大规模匹配 Kuhn 可能 TLE。

暴力/部分替代:

- $n \leq 20$ 可枚举每个点颜色判断冲突。
- 小规模匹配可枚举左部每个点选哪个右部。
- 最大匹配也可用 Dinic 建流网络, 代码更长但更稳。

升级方向:

- 大规模二分图匹配: Hopcroft-Karp 或 Dinic。
- 带权匹配: 费用流或 KM。
- 二分图最小点覆盖: 在最大匹配后用经典构造。

最小测试样例:

染色:

```

3 3
1 2
2 3
3 1
输出 NO

```

匹配:

```

nL=2,nR=2, edges: 1-1, 1-2, 2-2
最大匹配 = 2

```

GRAPH-08 有向图强连通分量 SCC

模块编号: GRAPH-08

模块名称: SCC、Tarjan 强连通分量

标签: [图论][SCC][Tarjan][有向图][缩点]

一句话用途: 把有向图中互相可达的点缩成一个强连通分量, 得到 DAG 后继续做 DP 或统计。

题面触发词:

- 有向图互相可达。
- 强连通分量、缩点。
- 一组点之间都能到达。
- 有向图中最少加边、缩点后入度出度。

什么时候用:

- 图是有向图。
- 需要判断哪些点互相可达。
- 有环的有向图要先缩成 DAG 再 DP。
- 需要统计 SCC 个数、每个分量大小。

不要什么时候用:

- 无向图连通块用 BFS/DFS 或 DSU。
- 只求从单点能到哪些点, 用普通 DFS/BFS。
- DAG 已经无环, 不需要 SCC。
- 递归深度极大时 Tarjan 递归可能爆栈, 要小心环境。

复杂度:

- $O(n + m)$ 时间。
- $O(n + m)$ 空间。

数据范围参考:

- $n, m \leq 2e5$ 常用。
- 递归栈风险与链长有关。

依赖的标准容器:

- Graph。
- 遍历 $G.g[u]$ 。
- 建图必须使用 $G.add_directed(u, v)$ 。
- 本模块只消费有向边, 混入无向边时会跳过。

输入如何整理:

```
Graph G(n);
for (int i = 1; i <= m; i++) {
    int u, v;
    cin >> u >> v;
    G.add_directed(u, v);
}
```

接口:

```
SCCResult tarjan_scc(const Graph& G)
Graph build_scc_dag(const Graph& G, const SCCResult& scc)
```

输出能力:

- $belong[u]$: 点 u 属于哪个 SCC。
- $size[c]$: 分量大小。
- cnt : 分量数量。
- 缩点 DAG。

下游可接:

- DAG DP。
- 缩点后拓扑排序。
- 入度/出度统计。

可拼接模块:

- Graph + SCC + Topo + DAG DP。
- SCC + component indegree/outdegree。

模板代码:

```
struct SCCResult {
    int cnt = 0;
    vector<int> belong;
    vector<int> sz;
};

struct TarjanSCC {
    const Graph *G = nullptr;
    int timer = 0, scc_cnt = 0;
    vector<int> dfn, low, in_st, st, belong, sz;

    void dfs(int u) {
        dfn[u] = low[u] = ++timer;
        st.push_back(u);
        in_st[u] = 1;
        for (auto e : G->g[u]) {
            if (!e.directed) continue;
            int v = e.to;
            if (!dfn[v]) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (in_st[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (low[u] == dfn[u]) {
            scc_cnt++;
            int x = 0;
            do {
                x = st.back();
                st.pop_back();
                in_st[x] = 0;
                belong[x] = scc_cnt;
                if ((int)sz.size() <= scc_cnt) sz.resize(scc_cnt + 1, 0);
                sz[scc_cnt]++;
            } while (x != u);
        }
    }

    SCCResult run(const Graph &graph) {
        G = &graph;
        int n = G->n;
        timer = scc_cnt = 0;
        dfn.assign(n + 1, 0);
        low.assign(n + 1, 0);
        in_st.assign(n + 1, 0);
        belong.assign(n + 1, 0);
        sz.assign(1, 0);
        st.clear();
        for (int i = 1; i <= n; i++) {
            if (!dfn[i]) dfs(i);
        }
        return {scc_cnt, belong, sz};
    }
};

SCCResult tarjan_scc(const Graph &G) {
    TarjanSCC solver;
    return solver.run(G);
}
```

```

Graph build_scc_dag(const Graph &G, const SCCResult &scc) {
    Graph dag(scc.cnt);
    set<pair<int, int>> seen;
    for (auto e : G.edges) {
        if (!e.directed) continue;
        int a = scc.belong[e.from];
        int b = scc.belong[e.to];
        if (a == b) continue;
        if (seen.insert({a, b}).second) {
            dag.add_directed(a, b, e.w);
        }
    }
    return dag;
}

```

```

pair<int, int> scc_zero_indeg_outdeg(const Graph &dag) {
    vector<int> indeg(dag.n + 1, 0), outdeg(dag.n + 1, 0);
    for (auto e : dag.edges) {
        if (!e.directed) continue;
        outdeg[e.from]++;
        indeg[e.to]++;
    }
    int zero_in = 0, zero_out = 0;
    for (int i = 1; i <= dag.n; i++) {
        if (indeg[i] == 0) zero_in++;
        if (outdeg[i] == 0) zero_out++;
    }
    return {zero_in, zero_out};
}

```

调用示例:

```

auto scc = tarjan_scc(G);
cout << scc.cnt << "\n";
auto dag = build_scc_dag(G, scc);
auto order = topo_sort(dag);

if (scc.cnt == 1) {
    cout << 0 << "\n";
} else {
    auto [zin, zout] = scc_zero_indeg_outdeg(dag);
    cout << max(zin, zout) << "\n"; // 最少加边使缩点 DAG 强连通
}

```

常见坑:

- SCC 是有向图概念, 建边要用 `G.add_directed(u, v)`。
- 标准模板会跳过无向边; 如果题目本身就是有向图, 不要混入无向边。
- `dfn/low/in_st` 三个数组不能混用。
- 弹栈时直到弹出 `u` 为止。
- 缩点后可能有重边, 本模板按 `(from, to)` 去重且保留第一条权值; 如果要最短路取最小权、最长路取最大权、路径计数保留重边, 需要按题意改去重策略。
- 递归深度过大可能爆栈。
- SCC 编号不保证拓扑序。

暴力/部分替代:

- $n \leq 300$ 可 Floyd 求可达矩阵, 互相可达归为一组。
- $n \leq 2000$ 可从每个点 DFS, 得到可达性后分组。
- 只判断两个点是否互达, 可以分别 DFS 一次。

升级方向:

- 缩点 DAG 后接 Topo/DAG DP。
- 统计最少加边强连通: 看缩点 DAG 入度为 0 和出度为 0 的分量数。
- 2-SAT 是 SCC 的专门应用。

最小测试样例：

```
4 4
1 2
2 1
2 3
3 4
SCC: {1,2}, {3}, {4}
缩点后是 DAG
```

GRAPH-09 树上 DFS、距离与 LCA

模块编号：GRAPH-09

模块名称：树上 DFS、树上距离、LCA

标签：[图论][树][DFS][LCA][倍增][距离]

一句话用途：在无向树上预处理深度、根距离和倍增父亲，快速回答祖先、最近公共祖先、两点距离。

题面触发词：

- 树、 n 个点 $n-1$ 条边。
- 祖先、公共祖先、最近公共祖先。
- 树上两点距离、路径长度。
- 多次询问树上路径。

什么时候用：

- 输入是一棵树或森林中的一棵根树。
- 多次询问 LCA 或两点距离。
- 边权可能存在，距离用 ll。
- 需要父节点、深度、到根距离。

不要什么时候用：

- 一般图有环时不能直接当树。
- 单次两点路径可 BFS/DFS 一次，不一定要 LCA。
- 动态加边删边的树路径不是本模板。
- 根会频繁变化时，需要额外处理。

复杂度：

- 预处理： $O(n \log n)$ 。
- 每次 LCA： $O(\log n)$ 。
- 空间： $O(n \log n)$ 。

数据范围参考：

- $n, q \leq 2e5$ 常规可用。
- LOG 自动按 n 计算。

依赖的标准容器：

- Graph。
- 树边用 `G.add_undirected(u, v, w)`。
- 遍历 `G.g[u]`。
- 调用 `build` 前最好先用 `TREE-01 is_connected_tree(T)` 检查：无向、连通、边数为 $n-1$ 。

输入如何整理：

```
Graph T(n);
for (int i = 1; i <= n - 1; i++) {
    int u, v;
    ll w;
    cin >> u >> v >> w;
    T.add_undirected(u, v, w);
}
```

接口：

```
LCA.build(Graph, root)
LCA.query(u, v)
LCA.distance(u, v)
depth[u], distRoot[u], up[k][u]
```

输出能力:

- 每个点深度。
- 每个点到根的距离。
- u, v 的 LCA。
- u, v 的树上距离。

下游可接:

- 树上路径查询。
- Kruskal 后建 MST 树再查路径。
- 树形 DP。

可拼接模块:

- Graph(Tree) + DFS + LCA。
- Kruskal + Tree + LCA。
- TreeDFS + DP。

模板代码:

```
struct LCA {
    int n = 0, LOG = 0;
    int root = 1;
    vector<int> depth;
    vector<ll> distRoot;
    vector<vector<int>> up;

    void build(const Graph &G, int root_ = 1) {
        // 前置条件: G 是无向连通树, root_ 是 1..n 的点号。
        n = G.n;
        root = root_;
        LOG = 1;
        while ((1 << LOG) <= n) LOG++;
        depth.assign(n + 1, 0);
        distRoot.assign(n + 1, 0);
        up.assign(LOG, vector<int>(n + 1, 0));

        queue<int> q;
        vector<int> vis(n + 1, 0);
        vis[root] = 1;
        up[0][root] = root;
        depth[root] = 1;
        q.push(root);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto e : G.g[u]) {
                int v = e.to;
                if (vis[v]) continue;
                vis[v] = 1;
                up[0][v] = u;
                depth[v] = depth[u] + 1;
                distRoot[v] = distRoot[u] + e.w;
                q.push(v);
            }
        }
        for (int k = 1; k < LOG; k++) {
            for (int v = 1; v <= n; v++) {
                up[k][v] = up[k - 1][up[k - 1][v]];
            }
        }
    }
};
```

```

}

int lift(int u, int steps) const {
    for (int k = 0; k < LOG; k++) {
        if (steps & (1 << k)) u = up[k][u];
    }
    return u;
}

int query(int a, int b) const {
    if (depth[a] < depth[b]) swap(a, b);
    a = lift(a, depth[a] - depth[b]);
    if (a == b) return a;
    for (int k = LOG - 1; k >= 0; k--) {
        if (up[k][a] != up[k][b]) {
            a = up[k][a];
            b = up[k][b];
        }
    }
    return up[0][a];
}

ll distance(int a, int b) const {
    int c = query(a, b);
    return distRoot[a] + distRoot[b] - 2LL * distRoot[c];
}
};

```

调用示例:

```

LCA lca;
lca.build(T, 1);
int c = lca.query(u, v);
cout << c << "\n";
cout << lca.distance(u, v) << "\n";

```

常见坑:

- 树要用无向边。
- n 个点必须有 $n-1$ 条边且连通, 森林要对每棵树单独处理或加超级根。
- 边权距离用 `ll`。
- 根的父亲设成根自己, 避免查询时跳到虚拟 0 点。
- 根不同, 父子关系不同, 但无权/带权两点距离不受根影响。
- 如果只读无权树, `w` 默认 1。

暴力/部分分替代:

- 单次距离查询可从 u BFS/DFS 到 v 。
- q 很小可每次爬父亲或 DFS。
- $n \leq 2000$ 可预处理任意两点距离。

升级方向:

- 路径上最大边: 倍增表同时存 `mx[k][v]`。
- 子树查询: DFS 序 + 树状数组/SegmentTree。
- 树上修改查询: 重链剖分, 低优先级。

最小测试样例:

```

5
1 2 3
1 3 2
2 4 4
2 5 1
LCA(4,5)=2
distance(4,3)=9

```

GRAPH-10 Dinic 最大流低优先级补充

模块编号: GRAPH-10

模块名称: Dinic 最大流

标签: [图论][网络流][Dinic][低优先级]

一句话用途: 当题目能建成容量网络时, 用 Dinic 求从源点到汇点的最大可发送流量。

题面触发词:

- 最大流、容量、源点、汇点。
- 每条边最多通过多少。
- 二分图匹配的大规模版本。
- 割、最少删除容量。

什么时候用:

- 题目明确是流量从源点流到汇点。
- 每条边有容量限制。
- 二分图匹配用普通匹配会超时, 想换最大流。
- 需要最小割值, 最大流等于最小割。

不要什么时候用:

- 普通最短路、MST、连通性不要用流。
- 有费用最小化不是 Dinic, 要最小费用最大流。
- 标准 Graph 不适合残量网络, 必须用独立 FlowGraph。
- 没想清楚建图时不要硬套。

复杂度:

- 常见竞赛规模表现较好。
- 理论上界较复杂, 纸质版只按“中等规模网络流”使用。

数据范围参考:

- 点边几千到几万视图结构和时限而定。
- 二分图匹配网络通常可用。

依赖的标准容器:

- 独立 FlowGraph。
- 不使用 Graph, 因为需要反向边和残量容量。
- LINF 依赖主骨架 `const ll LINF = 4'000'000'000'000'000'000LL;`。

输入如何整理:

```
FlowGraph F(n);
F.add_edge(u, v, cap);
// addEdge 只是旧题解兼容别名; 新写统一用 add_edge。
```

接口:

```
FlowGraph.init(n)
FlowGraph.add_edge(u, v, cap)
FlowGraph.addEdge(u, v, cap) // 旧题解兼容包装别名, 新写不推荐
FlowGraph.maxflow(s, t)
```

输出能力:

- 最大流值。
- 残量网络可进一步推出最小割集合。

下游可接:

- 二分图匹配。
- 最小割判定。
- 可行流进阶。

可拼接模块:

- Bipartite Matching -> Dinic。
- Binary Answer + Dinic Check。

模板代码:

```
struct FlowEdge {
    int to;
    int rev;
    ll cap;
};

struct FlowGraph {
    int n;
    vector<vector<FlowEdge>> g;
    vector<int> level, it;

    FlowGraph(int n = 0) {
        init(n);
    }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
    }

    void add_edge(int u, int v, ll cap) {
        FlowEdge a{v, (int)g[v].size(), cap};
        FlowEdge b{u, (int)g[u].size(), 0};
        g[u].push_back(a);
        g[v].push_back(b);
    }

    void addEdge(int u, int v, ll cap) {
        add_edge(u, v, cap);
    }

    bool bfs(int s, int t) {
        level.assign(n + 1, -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto e : g[u]) {
                if (e.cap > 0 && level[e.to] == -1) {
                    level[e.to] = level[u] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] != -1;
    }

    ll dfs(int u, int t, ll f) {
        if (u == t) return f;
        for (int &i = it[u]; i < (int)g[u].size(); i++) {
            FlowEdge &e = g[u][i];
            if (e.cap <= 0 || level[e.to] != level[u] + 1) continue;
            ll ret = dfs(e.to, t, min(f, e.cap));
            if (ret > 0) {
                e.cap -= ret;
                g[e.to][e.rev].cap += ret;
                return ret;
            }
        }
    }
}
```



```

        return 0;
    }

    ll maxflow(int s, int t) {
        ll flow = 0;
        while (bfs(s, t)) {
            it.assign(n + 1, 0);
            while (true) {
                ll f = dfs(s, t, LINF);
                if (f == 0) break;
                flow += f;
            }
        }
        return flow;
    }
};

```

调用示例:

```

FlowGraph F(n);
F.add_edge(s, a, cap1);
F.add_edge(a, t, cap2);
cout << F.maxflow(s, t) << "\n";

```

常见坑:

- Dinic 必须有反向边, 不能用标准 Graph 直接替代。
- add_edge 只加有向容量边; 无向容量边通常要加两条有向边。
- 容量和答案用 ll。
- dfs 找到流后要同时改正向和反向容量。
- 多组数据要 init(n)。
- 递归深度过大时可能有栈风险。

暴力/部分分替代:

- 小图可枚举割集或路径增广。
- 二分图匹配小规模用 Kuhn。
- 容量都为 1 且图小, 可每次 BFS 找增广路。

升级方向:

- 有费用: 最小费用最大流。
- 只求二分图最大匹配: Hopcroft-Karp 更专门。
- 需要割边集合: 最大流后从源点在残量网络 DFS 标记可达点。

最小测试样例:

```

s=1,t=4
1->2 cap 3
1->3 cap 2
2->4 cap 2
3->4 cap 4
最大流 = 4

```

GRAPH-11: 图/树部分分作战表

模块编号: GRAPH-11

模块名称: 图/树部分分作战表

标签: 图论、树、部分分、暴力、BFS、Floyd、LCA、状态搜索

一句话用途: 图/树题不会正解时, 按 V0 到 V5 逐步写, 至少拿小数据、特殊性质和合法输出分。

题面触发词:

- 图、树、连通、最短路、生成树、LCA、子树、路径。
- 数据有多个子任务或特殊性质。

- 每题多次提交取最高分。

什么时候用：

- 图/树正解一时想不出。
- 想先写一个稳定小数据版本。
- 题面存在 $n \leq 20/200/2000$ 、链、星、边权全 1、树等特殊条件。

不要什么时候用：

- 已经能明确套 BFS/Dijkstra/Kruskal/LCA/树 DP 时，直接写正解。
- 输出必须严格最优且无部分分时，不要停在兜底版本。

复杂度：

- V0/V1: $O(n+m)$ 或只读入输出。
- V2: Floyd $O(n^3)$ ，每问 DFS/BFS $O(n+m)$ 。
- V3: 枚举点/边集常见 $O(2^n)$ 或 $O(2^m)$ ，只适合小数据。
- V4: 树上每问 DFS $O(nq)$ ，暴力 LCA $O(nq)$ 最坏。
- V5: 正解模块复杂度见各模块。

数据范围信号：

- $n \leq 20$: 枚举点集/边集。
- $n \leq 200$: Floyd、小图邻接矩阵。
- $n, q \leq 2000$: 每问 DFS/BFS、暴力 LCA。
- $m = n - 1$: 优先判断是否是树。
- 边权全 1: BFS 替代 Dijkstra。

依赖的标准容器：

- Graph
- DSU
- queue
- `vector<vector<int>>`

输入如何整理：

- 统一先读成 Graph G。
- 记录特殊性质: `is_tree`、`all_weight_one`、`all_directed`、`n_small`。
- 树题优先构造 TreeInfo。

接口：

V0 合法兜底 -> V1 特判 -> V2 小图精确 -> V3 枚举 -> V4 树暴力 -> V5 正解

输出能力：

- 合法输出。
- 小数据精确解。
- 特殊性质精确解。
- 正解升级路线。

下游可接：

- GRAPH-01/02/03/04/05/06/09/10。
- TREE-01。
- DP-14/15。
- BRUTE-11 状态 BFS。

可拼接模块：

- Floyd: 小图最短路。
- DSU: 连通性/MST/生成树检查。
- BFS/DFS: 连通、最短步数、树上路径。
- TreeInfo: 树上父亲/深度/DFS 序。

V0: 读入完整 + 合法兜底

先保证：

1. 输入全部读完。
2. 输出格式合法。

3. $n=1$ 、 $m=0$ 、 $q=0$ 不 RE。
4. 不输出越界点号或不存在边号。

常见兜底:

题型	兜底
判断连通	无边且 $n>1$ 输出 No, $n=1$ 输出 Yes
最短路	不会时至少不可达输出 -1, 不要乱输出路径
构造路径	若 $s==t$, 输出长度 0 或单点路径
输出边集	空边集是否允许先看题面, 不允许就输出合法失败标志
树题	$n==1$ 单独处理

V1: 特殊性质特判

```
bool all_weight_one = true;
bool maybe_tree = ((int)G.edges.size() == G.n - 1);
for (auto e : G.edges) {
    if (e.w != 1) all_weight_one = false;
}
```

特殊性质	写法
边权全 1	BFS
小图	Floyd
已是树	TreeInfo/LCA/树 DP
链	转数组前缀和/相邻处理
星	中心点特判
DAG	Topo + DP

V2: 小图精确

小图多源最短路:

```
const int MAXF = 505;
static ll dist_small[MAXF][MAXF];

void floyd_small(const Graph& G) {
    int n = G.n;
    assert(n > 0 && n < MAXF);
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            dist_small[i][j] = (i == j ? 0 : LINF);
        }
    }
    for (auto e : G.edges) {
        dist_small[e.from][e.to] = min(dist_small[e.from][e.to], e.w);
        if (!e.directed) dist_small[e.to][e.from] = min(dist_small[e.to][e.from], e.w);
    }
    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            if (dist_small[i][k] == LINF) continue;
            for (int j = 1; j <= n; j++) {
                if (dist_small[k][j] == LINF) continue;
                dist_small[i][j] = min(dist_small[i][j], dist_small[i][k] + dist_small[k][j]);
            }
        }
    }
}
```

每问 BFS: 仅限无权图或边权全为 1:

```
int bfs_one_query(const Graph& G, int s, int t) {
    vector<int> dist(G.n + 1, -1);
    queue<int> q;
```

```

dist[s] = 0;
q.push(s);
while (!q.empty()) {
    int u = q.front();
    q.pop();
    if (u == t) return dist[u];
    for (auto e : G.g[u]) {
        int v = e.to;
        if (dist[v] != -1) continue;
        dist[v] = dist[u] + 1;
        q.push(v);
    }
}
return -1;
}

```

V3: 枚举点集/边集

枚举点集常用于：

- 最大独立集小数据。
- 最小点覆盖小数据。
- 选关键点是否满足条件。

枚举边集常用于小图 MST：

```

ll brute_mst_edges(int n, vector<FullEdge>& edges) {
    int m = (int)edges.size();
    if (m > 22) return LINF;
    ll ans = LINF;
    for (long long mask = 0; mask < (1LL << m); mask++) {
        if (__builtin_popcount((unsigned)mask) != n - 1) continue;
        DSU dsu(n);
        ll cost = 0;
        int used = 0;
        for (int i = 0; i < m; i++) {
            if (!(mask >> i & 1)) continue;
            if (edges[i].directed) continue;
            if (dsu.unite(edges[i].from, edges[i].to)) {
                cost += edges[i].w;
                used++;
            }
        }
        if (used == n - 1) ans = min(ans, cost);
    }
    return ans == LINF ? -1 : ans;
}

```

只适合 $m \leq 22$ 左右。

V4: 树题暴力

每问 DFS 找距离：

```

bool dfs_path_distance(const Graph& T, int u, int p, int target, ll cur, ll& ans) {
    if (u == target) {
        ans = cur;
        return true;
    }
    for (auto e : T.g[u]) {
        int v = e.to;
        if (v == p) continue;
        if (dfs_path_distance(T, v, u, target, cur + e.w, ans)) return true;
    }
    return false;
}

```

```

11 tree_distance_one_query(const Graph& T, int u, int v) {
    11 ans = -1;
    dfs_path_distance(T, u, 0, v, 0, ans);
    return ans;
}

```

暴力爬父亲 LCA:

```

int lca_climb(int u, int v, const vector<int>& parent, const vector<int>& depth) {
    while (depth[u] > depth[v]) u = parent[u];
    while (depth[v] > depth[u]) v = parent[v];
    while (u != v) {
        u = parent[u];
        v = parent[v];
    }
    return u;
}

```

$n, q \leq 2000$ 时可以先交。大数据再换 GRAPH-09 倍增 LCA。

V5: 升级路线表

小数据/特殊版	正解模块
每问 BFS 无权最短路	GRAPH-02 BFS, 必要时多源 BFS
每问 Dijkstra	GRAPH-03 Dijkstra
Floyd 小图	Dijkstra/Floyd 按数据范围取舍
枚举边集 MST	GRAPH-06 Kruskal
暴力拓扑 DFS	GRAPH-05 Topo / DP-15 DAG DP
每问树上 DFS	TREE-01 + GRAPH-09 LCA
枚举点集树 DP	DP-14 树形 DP
普通 BFS 状态	BRUTE-11 状态 BFS

常见坑:

- 小数据版本也要保证大数据不会 TLE 到完全没结果; 可用条件分支保护。
- 有向/无向别建反。
- 边权全 1 才能 BFS。
- 树必须连通且 $m=n-1$ 。
- 枚举 $1 \leq m$ 时 m 不能太大。
- 暴力 DFS 树路径递归深度可能爆栈, 链很长时谨慎。

暴力/部分替代:

- 本模块本身就是部分分路线。
- 不会正解时, 至少保留小数据精确解和特殊性质精确解。

升级方向:

合法兜底

- > 特判特殊性质
- > 小图精确
- > 暴力枚举/每问 DFS
- > 正式图论/树算法

最小测试样例:

树路径:

```

3
1 2 5
2 3 7
query 1 3
输出: 12

```

GRAPH-12 0-1 BFS 与分层图最短路

模块编号: GRAPH-12

模块名称: 0-1 BFS、分层图、带次数限制的状态最短路

标签: [图论][最短路][0-1 BFS][分层图][状态图][deque]

一句话用途: 边权只有 0/1 时用 deque 代替 Dijkstra; 题目有 “最多用 k 次免费/折扣/跳过/特殊能力” 时, 把 次数 used 加进状态做分层最短路。

题面触发词:

- 边权只有 0 或 1。
- “免费通过最多 k 次” “最多改 k 条边” “最多跳过 k 次费用”。
- “状态里除了点编号, 还要记住用了几次机会”。
- 网格或图上求最小代价, 但代价不是普通单一顶点。

什么时候用:

- 0-1 BFS: 所有边权都在 $\{0, 1\}$ 。
- 分层图: 到达同一个点时, “已经使用几次机会” 会影响未来。
- 普通 $\text{dist}[u]$ 不够, 必须用 $\text{dist}[\text{used}][u]$ 。

不要什么时候用:

- 边权不是 0/1, 且有一般非负权, 优先 Dijkstra。
- 有负权边, 转 Bellman-Ford/SPFA。
- 状态维度很多且 $n \times \text{状态数}$ 爆内存时, 不要硬开二维数组。
- 只是无权最少步数, 普通 BFS 更简单。

复杂度:

- 0-1 BFS: $O(n + m)$ 。
- 分层图若每层边仍是 0/1 且用 deque: $O((k+1) * (n+m))$ 。
- 分层图若有一般非负权且用 priority_queue: $O((k+1) * (n+m) * \log((k+1)*n))$ 。

数据范围参考:

- $n, m \leq 2e5$ 且 $k \leq 20/50$ 常见可做。
- 如果 k 接近 n, 先估算 $(k+1)*(n+1)$ 是否会爆内存。

依赖的标准容器:

- 标准 Graph。
- `deque<int>`。
- `static ll layer_dist[LAYER_MAXK][LAYER_MAXN]`, 点号仍是 $1..n$, 名字加前缀避免和其他模块的 MAXN/MAXK 冲突。
- 依赖主骨架: `using ll = long long; const ll LINF = 4'000'000'000'000'000'000LL;`

输入如何整理:

```
Graph G(n);
for (int i = 1; i <= m; i++) {
    int u, v;
    ll w;
    cin >> u >> v >> w; // w 必须是 0 或 1 才能 0-1 BFS
    G.add_undirected(u, v, w);
}
```

接口:

```
vector<ll> zero_one_bfs(const Graph& G, int s);
bool shortest_with_free_passes(const Graph& G, int s, int k);
```

输出能力:

- $\text{dist}[v]$: 0-1 权图从 s 到 v 的最短路。
- $\text{dist}[\text{used}][v]$: 从 s 到 v, 已经使用 used 次特殊机会时的最小代价。
- 最终答案通常是 $\min(\text{dist}[0..k][t])$ 。

下游可接:

- 状态 BFS。
- Dijkstra。
- DP-03B 状态增维: used 是典型 “后效性吸收到状态里” 的维度。

可拼接模块：

- Graph + 0-1 BFS.
- Graph + dist[used][u] + Dijkstra/0-1 BFS.
- GridState + used.

模板代码：

```
vector<ll> zero_one_bfs(const Graph &G, int s) {
    vector<ll> dist(G.n + 1, LINF);
    deque<int> dq;
    dist[s] = 0;
    dq.push_front(s);

    while (!dq.empty()) {
        int u = dq.front();
        dq.pop_front();
        for (auto e : G.g[u]) {
            int v = e.to;
            ll w = e.w;
            assert(w == 0 || w == 1);
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                if (w == 0) dq.push_front(v);
                else dq.push_back(v);
            }
        }
    }
    return dist;
}

const int LAYER_MAXN = 200000 + 5;
const int LAYER_MAXK = 55;
static ll layer_dist[LAYER_MAXK][LAYER_MAXN];

bool shortest_with_free_passes(const Graph &G, int s, int k) {
    if (k < 0 || k >= LAYER_MAXK || G.n <= 0 || G.n >= LAYER_MAXN) return false;
    for (int used = 0; used <= k; used++) {
        for (int v = 1; v <= G.n; v++) {
            layer_dist[used][v] = LINF;
        }
    }
    priority_queue<tuple<ll, int, int>, vector<tuple<ll, int, int>>, greater<tuple<ll, int, int>>> pq;

    layer_dist[0][s] = 0;
    pq.push({0, 0, s}); // cost, used, node

    while (!pq.empty()) {
        auto [d, used, u] = pq.top();
        pq.pop();
        if (d != layer_dist[used][u]) continue;

        for (auto e : G.g[u]) {
            int v = e.to;
            ll w = e.w;

            if (d + w < layer_dist[used][v]) {
                layer_dist[used][v] = d + w;
                pq.push({layer_dist[used][v], used, v});
            }

            if (used < k && d < layer_dist[used + 1][v]) {
                layer_dist[used + 1][v] = d; // 免费走这条边
                pq.push({layer_dist[used + 1][v], used + 1, v});
            }
        }
    }
}
```

```

    }
}
}
return true;
}

```

调用示例:

```

auto dist01 = zero_one_bfs(G, 1);
cout << (dist01[n] == LINF ? -1 : dist01[n]) << '\n';

int k = 2;
if (!shortest_with_free_passes(G, 1, k)) {
    cout << "-1\n";
    return;
}
ll ans = LINF;
for (int used = 0; used <= k; used++) ans = min(ans, layer_dist[used][n]);
cout << (ans == LINF ? -1 : ans) << '\n';

```

常见坑:

- 0-1 BFS 只能处理边权 0/1, 不是 “小整数 BFS”。
- 0 权边要 push_front, 1 权边要 push_back。
- 分层图答案常常是 min(dist[used][t]), 不是只看 dist[k][t]。
- dist[used][u] 中 used 必须表示已经用了多少次机会, 含义不要混。
- 有向图要用 add_directed, 无向图要用 add_undirected。
- k*n 太大时, 改用 unordered_map 稀疏记忆化可能拿部分分。

暴力/部分分替代:

- n <= 20: DFS 枚举是否使用免费机会, 记录最小值。
- k = 0: 直接普通 Dijkstra。
- 边权全 1: 普通 BFS。
- 小图任意两点: Floyd 后再做简单枚举。

升级方向:

- 普通 BFS -> 0-1 BFS -> Dijkstra。
- dist[u] 不够 -> dist[used][u]。
- 网格带钥匙/体力/次数 -> dist[x][y][state]。

最小测试样例:

分层 Dijkstra / 免费一次样例:

```

3 3
1 2 1
2 3 1
1 3 5
k=1

```

免费一次后可直接免费走 1->3, 最小代价是 0

GRAPH-13 无向图桥与割点

模块编号: GRAPH-13

模块名称: Lowlink、桥、割点

标签: [图论][无向图][桥][割点][lowlink][Tarjan]

一句话用途: 找无向图中删掉会增加连通块数量的边或点, 用于 “关键道路、关键城市、网络脆弱点” 等题。

题面触发词:

- 关键边、桥、割边。
- 关键点、割点。
- 删除一条边/一个点后是否不连通。
- 网络可靠性、必经道路。

什么时候用：

- 图是无向图。
- 需要一次性找出所有桥或割点。
- n, m 较大，不能枚举删除每条边/每个点再 DFS。

不要什么时候用：

- 有向图强连通问题，转 SCC。
- 只是判断普通连通性，用 DFS/BFS/DSU。
- 动态加删边，不是本模板。
- 递归深度可能到 $2e5$ 时要注意栈风险。

复杂度：

- $O(n + m)$ 时间。
- $O(n + m)$ 空间。

数据范围参考：

- $n, m \leq 2e5$ 常见可用。
- 链状图会有深递归风险；若评测栈很小，考虑改迭代或只拿部分分。

依赖的标准容器：

- 标准 Graph。
- 图必须用无向边 `add_undirected`。
- 依赖 `AdjEdge.edge_index` 区分重边，不能只用 `v == parent` 跳过。

输入如何整理：

```
Graph G(n);
for (int i = 1; i <= m; i++) {
    int u, v;
    cin >> u >> v;
    G.add_undirected(u, v);
}
```

接口：

```
LowlinkResult find_bridges_cutpoints(const Graph& G);
```

输出能力：

- `bridge_input_ids`：桥对应的题面边号，1-index，可直接输出。
- `bridges`：桥的两个端点。
- `is_cut[u]`：点 u 是否为割点。
- `dfn/low`：调试或进阶使用。

下游可接：

- 桥边删除后分块。
- 边双连通/点双连通低优先级进阶。
- 图论部分分：小图先枚举删除，满分再 Lowlink。

可拼接模块：

- Graph + Lowlink。
- Lowlink + DSU。
- Lowlink + DFS component。

模板代码：

```
struct LowlinkResult {
    vector<int> dfn;
    vector<int> low;
    vector<int> is_cut;
    vector<int> bridge_input_ids;
    vector<pair<int, int>> bridges;
};

struct LowlinkSolver {
    const Graph *G = nullptr;
```

```

int timer = 0;
LowlinkResult res;

void dfs(int u, int parent_edge) {
    res.dfn[u] = res.low[u] = ++timer;
    int child_count = 0;

    for (auto e : G->g[u]) {
        if (e.directed) continue;
        int v = e.to;
        if (!res.dfn[v]) {
            child_count++;
            dfs(v, e.edge_index);
            res.low[u] = min(res.low[u], res.low[v]);

            if (res.low[v] > res.dfn[u]) {
                res.bridge_input_ids.push_back(e.input_id);
                res.bridges.push_back({u, v});
            }

            if (parent_edge != -1 && res.low[v] >= res.dfn[u]) {
                res.is_cut[u] = 1;
            }
        } else if (e.edge_index != parent_edge) {
            res.low[u] = min(res.low[u], res.dfn[v]);
        }
    }

    if (parent_edge == -1 && child_count > 1) {
        res.is_cut[u] = 1;
    }
}

LowlinkResult run(const Graph &graph) {
    G = &graph;
    timer = 0;
    res.dfn.assign(G->n + 1, 0);
    res.low.assign(G->n + 1, 0);
    res.is_cut.assign(G->n + 1, 0);
    res.bridge_input_ids.clear();
    res.bridges.clear();

    for (int i = 1; i <= G->n; i++) {
        if (!res.dfn[i]) dfs(i, -1);
    }
    return res;
}

};

LowlinkResult find_bridges_cutpoints(const Graph &G) {
    LowlinkSolver solver;
    return solver.run(G);
}

调用示例:

auto res = find_bridges_cutpoints(G);
cout << res.bridges.size() << '\n';
for (auto [u, v] : res.bridges) {
    cout << u << ' ' << v << '\n';
}
for (int u = 1; u <= n; u++) {
    if (res.is_cut[u]) cout << "cut " << u << '\n';
}

```

常见坑:

- 桥/割点是无向图概念, 建边要用 `G.add_undirected(u, v)`, 模板只处理无向边。
- 有重边时不能只写 `if (v == parent) continue`, 要按边 id 跳过父边。
- 根节点是割点的条件是 DFS 树儿子数大于 1。
- `low[v] > dfn[u]` 是桥; `low[v] >= dfn[u]` 是非根割点条件。
- `bridge_input_ids` 是 1-index 题面边号; 如果只需要输出桥的两个端点, 直接用 `bridges`。
- 递归深度很深可能栈溢出。

暴力/部分替代:

- `n, m <= 2000`: 枚举删除每条边, 再 DFS 判断连通。
- `n <= 500`: 枚举删除每个点, 再 DFS 判断连通。
- 只问某一条边是不是桥: 删掉它后从一端 DFS 看能否到另一端。

升级方向:

- 桥 -> 边双连通分量。
- 割点 -> 点双连通分量。
- 有向“关键可达结构”通常转 SCC 或支配树, 支配树低优先级。

最小测试样例:

```
5 5
1 2
2 3
3 1
3 4
4 5
```

桥: 3-4, 4-5

割点: 3, 4

TREE-00 二叉树基础

模块编号: TREE-00

模块名称: Binary Tree 数组建树、遍历与重建

标签: [树][二叉树][遍历][递归][栈][重建]

一句话用途: 用 `lc[u]` / `rc[u]` 表示左右儿子, 快速完成二叉树建树、前中后序遍历和由前序 + 中序重建。

题面触发词:

- 二叉树、左儿子、右儿子。
- 前序遍历、中序遍历、后序遍历。
- 给出每个点的左右孩子。
- 给出前序和中序, 求后序或还原树。
- 完全二叉树、堆式编号。

什么时候用:

- 节点编号是 $1..n$, 空儿子用 0 或 -1 表示。
- 题目明确是二叉树, 不需要一般图建边。
- 只需要遍历顺序、父亲、深度、子树大小等基础信息。

不要什么时候用:

- 输入是普通无向树, 每个点儿子数不固定, 应使用邻接表 DFS。
- 二叉搜索树插入/删除要按 BST 性质单独处理。
- 节点值不唯一时, 不能直接用“前序 + 中序”唯一重建。

复杂度:

- 建树: $O(n)$ 。
- 递归/迭代遍历: $O(n)$ 。
- 前序 + 中序重建: $O(n)$, 需要值唯一并建立位置表。

数据范围参考:

- `n <= 2e5`: 数组开 `N = 200000 + 5`。

- 深链递归可能爆栈, n 很大时遍历优先用迭代版。
- 完全二叉树编号: 左儿子 $2*u$, 右儿子 $2*u+1$ 。

依赖的标准容器:

- 1-index 数组: $lc[u]$ 、 $rc[u]$ 、 $fa[u]$ 。
- `vector<int>` 存遍历结果。
- `stack<int>` 做迭代遍历。

输入如何整理:

// 常见输入: 第 i 行给 i 的左儿子、右儿子, 0 表示空。

```
cin >> n;
for (int i = 1; i <= n; i++) {
    cin >> lc[i] >> rc[i];
}
```

接口:

`find_root()` -> 根据入度找根

`build_complete(n)` -> 按完全二叉树编号补左右儿子

`preorder_dfs(root)`, `inorder_dfs(root)`, `postorder_dfs(root)`

`preorder_iter(root)`, `inorder_iter(root)`, `postorder_iter(root)`

`build(preL, preR, inL, inR)` -> 前序 + 中序重建

输出能力:

- 前序、中序、后序遍历序列。
- 每个点父亲。
- 完全二叉树左右儿子。
- 由前序 + 中序得到左右儿子或后序。

下游可接:

- 树形 DP。
- DFS 序。
- 二叉树表达式求值。
- 二叉搜索树判断。

可拼接模块:

- TREE-00 + DP-14 TreeDP。
- TREE-00 + Stack。
- TREE-00 + GRAPH-09 LCA, 仅当二叉树转成普通树后需要路径查询。

模板代码: 数组建二叉树与遍历

```
#include <bits/stdc++.h>
using namespace std;

const int N = 200000 + 5;

int n, root;
int lc[N], rc[N], fa[N];

void clear_tree(int n) {
    for (int i = 0; i <= n; i++) {
        lc[i] = rc[i] = fa[i] = 0;
    }
}

int find_root() {
    vector<int> indeg(n + 1, 0);
    for (int u = 1; u <= n; u++) {
        if (lc[u]) {
            indeg[lc[u]]++;
            fa[lc[u]] = u;
        }
        if (rc[u]) {
            indeg[rc[u]]++;
        }
    }
    for (int i = 1; i <= n; i++) {
        if (indeg[i] == 0) {
            root = i;
            break;
        }
    }
}
```

```

        fa[rc[u]] = u;
    }
}

for (int i = 1; i <= n; i++) {
    if (indeg[i] == 0) return i;
}
return 0;
}

void read_left_right_tree() {
    cin >> n;
    clear_tree(n);
    for (int i = 1; i <= n; i++) {
        cin >> lc[i] >> rc[i];
        if (lc[i] < 0) lc[i] = 0;
        if (rc[i] < 0) rc[i] = 0;
    }
    root = find_root();
}

void build_complete(int n_) {
    n = n_;
    clear_tree(n);
    for (int u = 1; u <= n; u++) {
        if (2 * u <= n) {
            lc[u] = 2 * u;
            fa[2 * u] = u;
        }
        if (2 * u + 1 <= n) {
            rc[u] = 2 * u + 1;
            fa[2 * u + 1] = u;
        }
    }
    root = (n == 0 ? 0 : 1);
}

vector<int> pre_rec, in_rec, post_rec;

void preorder_dfs(int u) {
    if (u == 0) return;
    pre_rec.push_back(u);
    preorder_dfs(lc[u]);
    preorder_dfs(rc[u]);
}

void inorder_dfs(int u) {
    if (u == 0) return;
    inorder_dfs(lc[u]);
    in_rec.push_back(u);
    inorder_dfs(rc[u]);
}

void postorder_dfs(int u) {
    if (u == 0) return;
    postorder_dfs(lc[u]);
    postorder_dfs(rc[u]);
    post_rec.push_back(u);
}

vector<int> preorder_iter(int root) {
    vector<int> res;
    if (root == 0) return res;
    stack<int> st;

```

```

    st.push(root);
    while (!st.empty()) {
        int u = st.top();
        st.pop();
        res.push_back(u);
        if (rc[u]) st.push(rc[u]);
        if (lc[u]) st.push(lc[u]);
    }
    return res;
}

vector<int> inorder_iter(int root) {
    vector<int> res;
    stack<int> st;
    int u = root;
    while (u || !st.empty()) {
        while (u) {
            st.push(u);
            u = lc[u];
        }
        u = st.top();
        st.pop();
        res.push_back(u);
        u = rc[u];
    }
    return res;
}

vector<int> postorder_iter(int root) {
    vector<int> res;
    if (root == 0) return res;
    stack<pair<int, int>> st;
    st.push({root, 0});
    while (!st.empty()) {
        auto [u, visited] = st.top();
        st.pop();
        if (u == 0) continue;
        if (visited) {
            res.push_back(u);
        } else {
            st.push({u, 1});
            st.push({rc[u], 0});
            st.push({lc[u], 0});
        }
    }
    return res;
}

void print_vector(const vector<int> &v) {
    for (int i = 0; i < (int)v.size(); i++) {
        if (i) cout << ' ';
        cout << v[i];
    }
    cout << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    read_left_right_tree();

    print_vector(preorder_iter(root));
}

```

```

    print_vector(inorder_iter(root));
    print_vector(postorder_iter(root));
    return 0;
}

```

模板代码：由前序 + 中序重建

拼接提醒：下面是一个独立完整模板，所以重新声明了 N/n/lc/rc。如果已经复制了上面的二叉树基础模板，合并时复用同一套 N/n/lc/rc，只额外加入 pre/inord/pos/build/print_postorder 即可。

```

#include <bits/stdc++.h>
using namespace std;

const int N = 200000 + 5;

int n;
int pre[N], inord[N], pos[N];
int lc[N], rc[N];

int build(int pl, int pr, int il, int ir) {
    if (pl > pr) return 0;
    int u = pre[pl];
    int k = pos[u];
    int left_cnt = k - il;

    lc[u] = build(pl + 1, pl + left_cnt, il, k - 1);
    rc[u] = build(pl + left_cnt + 1, pr, k + 1, ir);
    return u;
}

void print_postorder(int u) {
    if (u == 0) return;
    print_postorder(lc[u]);
    print_postorder(rc[u]);
    cout << u << ' ';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n;
    for (int i = 1; i <= n; i++) cin >> pre[i];
    for (int i = 1; i <= n; i++) {
        cin >> inord[i];
        pos[inord[i]] = i;
    }

    int root = build(1, n, 1, n);
    print_postorder(root);
    cout << '\n';
    return 0;
}

```

调用示例：

```

root = find_root();
vector<int> a = preorder_iter(root);
vector<int> b = inorder_iter(root);
vector<int> c = postorder_iter(root);

```

常见坑：

- 空儿子统一转成 0，不要一会儿用 -1 一会儿用 0。
- 前序 + 中序重建要求节点值唯一；值重复时不能唯一确定。
- 如果节点值不是 1..n，pos[value] 数组要改成 unordered_map<int,int>。

- 递归遍历遇到链状树可能爆栈， $n \geq 2e5$ 时优先用迭代遍历。
- 找根时检查入度，不能默认 1 一定是根。
- 完全二叉树编号只适用于题目明确按堆式编号存树。

暴力/部分替代：

- $n \leq 2000$ ：递归遍历最省事，先拿基础分。
- 只问一次遍历：不必建复杂结构，读完 lc/rc 后直接 DFS。
- 前序 + 中序不会重建：可以递归切区间直接输出后序，不一定显式存树。

升级方向：

- 二叉树基础遍历 -> 树形 DP。
- 二叉树转普通树 -> LCA/树上距离。
- 多次子树查询 -> DFS 序 + 树状数组/SegmentTree。

最小测试样例：

输入左右儿子：

```
5
2 3
4 5
0 0
0 0
0 0
```

前序：1 2 4 5 3

中序：4 2 5 1 3

后序：4 5 2 3 1

TREE-01：普通树 RootedTree 信息层

模块编号：TREE-01

模块名称：普通树 RootedTree 信息层

标签：树、DFS 序、父亲、深度、子树大小、1-index、模块拼接

一句话用途：把一棵无向树整理成有根树信息：parent/depth/distRoot/order/tin/tout/subtree_size/children，后续接 LCA、树形 DP、DFS 序子树查询。

题面触发词：

- “给一棵树”
- “以 1 为根”
- “子树”
- “父亲、深度、祖先”
- “树上路径/树上 DP/子树查询”

什么时候用：

- 题目给的是普通无向树，需要先确定根。
- 后续要用树形 DP、LCA、DFS 序、子树大小。
- 想把树题的第一步统一成一个可复用结构。

不要什么时候用：

- 输入不是树，而是一般图有环。
- 动态加边删边，普通 DFS 序会失效。
- 只是单次连通性或最短路，不一定需要完整 TreeInfo。

复杂度：

- 建树信息： $O(n)$ 。
- 空间： $O(n)$ 。

数据范围信号：

- $n \leq 2e5$ 可用 BFS/迭代 DFS，避免递归爆栈。
- n 很小也可直接递归 DFS，但本模块给迭代版。

依赖的标准容器:

- 标准 Graph。
- `vector<int>`
- `vector<ll>`

输入如何整理:

```
Graph T(n);
for (int i = 1; i <= n - 1; i++) {
    int u, v;
    cin >> u >> v;
    T.add_undirected(u, v);
}
```

接口:

```
TreeInfo build_tree_info(const Graph& T, int root);
bool is_connected_tree(const Graph& G);
```

输出能力:

- `parent[u]`: 父亲。
- `depth[u]`: 深度。
- `distRoot[u]`: 到根距离。
- `children[u]`: 有根树儿子。
- `order`: 从根 BFS/DFS 的顺序。
- `tin/tout`: DFS 序, 子树区间是 `[tin[u], tout[u]]`。
- `subtree_size[u]`: 子树大小。

下游可接:

- GRAPH-09 LCA。
- DP-14 树形 DP。
- 树状数组/SegmentTree 子树查询。
- 树直径、换根 DP。

可拼接模块:

- Graph 标准建图。
- 树状数组: 对子树区间加/查。
- SegmentTree: 对子树区间最值/和。

模板代码:

```
struct TreeInfo {
    int n = 0;
    int root = 1;
    vector<int> parent;
    vector<int> depth;
    vector<ll> distRoot;
    vector<vector<int>> children;
    vector<int> order;
    vector<int> tin;
    vector<int> tout;
    vector<int> subtree_size;
};

bool is_connected_tree(const Graph& G) {
    if ((int)G.edges.size() != G.n - 1) return false;
    for (auto e : G.edges) {
        if (e.directed) return false;
    }
    vector<int> vis(G.n + 1, 0);
    queue<int> q;
    q.push(1);
    vis[1] = 1;
    int cnt = 0;
    while (!q.empty()) {
```

```

    int u = q.front();
    q.pop();
    cnt++;
    for (auto e : G.g[u]) {
        int v = e.to;
        if (vis[v]) continue;
        vis[v] = 1;
        q.push(v);
    }
}
return cnt == G.n;
}

TreeInfo build_tree_info(const Graph& T, int root = 1) {
    TreeInfo info;
    info.n = T.n;
    info.root = root;
    info.parent.assign(T.n + 1, 0);
    info.depth.assign(T.n + 1, 0);
    info.distRoot.assign(T.n + 1, 0);
    info.children.assign(T.n + 1, {});
    info.tin.assign(T.n + 1, 0);
    info.tout.assign(T.n + 1, 0);
    info.subtree_size.assign(T.n + 1, 1);

    queue<int> q;
    vector<int> vis(T.n + 1, 0);
    q.push(root);
    vis[root] = 1;
    info.parent[root] = 0;
    info.depth[root] = 0;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        info.order.push_back(u);
        for (auto e : T.g[u]) {
            int v = e.to;
            if (vis[v]) continue;
            vis[v] = 1;
            info.parent[v] = u;
            info.depth[v] = info.depth[u] + 1;
            info.distRoot[v] = info.distRoot[u] + e.w;
            info.children[u].push_back(v);
            q.push(v);
        }
    }

    int timer = 0;
    vector<int> it(T.n + 1, 0);
    vector<int> st;
    st.push_back(root);
    while (!st.empty()) {
        int u = st.back();
        if (it[u] == 0) {
            info.tin[u] = ++timer;
        }
        if (it[u] < (int)info.children[u].size()) {
            int v = info.children[u][it[u]++];
            st.push_back(v);
        } else {
            info.tout[u] = timer;
            st.pop_back();
        }
    }
}

```

```

    }
}

for (int idx = (int)info.order.size() - 1; idx >= 0; idx--) {
    int u = info.order[idx];
    info.subtree_size[u] = 1;
    for (int v : info.children[u]) {
        info.subtree_size[u] += info.subtree_size[v];
    }
}

return info;
}

调用示例:

if (!is_connected_tree(T)) {
    cout << "Not a tree\n";
    return;
}

TreeInfo tr = build_tree_info(T, 1);
cout << tr.parent[u] << ' ' << tr.depth[u] << ' ' << tr.subtree_size[u] << '\n';

```

DFS 序子树查询接法:

```

// u 的子树对应 DFS 序闭区间 [tin[u], tout[u]]
BIT fw;
fw.init(n);
for (int u = 1; u <= n; u++) {
    fw.add(tr.tin[u], value[u]);
}
ll subtree_sum = fw.query(tr.tin[u], tr.tout[u]);

```

二叉树转普通 Graph:

```

Graph binary_tree_to_graph(int n, const vector<int>& lc, const vector<int>& rc) {
    Graph T(n);
    for (int u = 1; u <= n; u++) {
        if (lc[u] > 0) T.add_undirected(u, lc[u]);
        if (rc[u] > 0) T.add_undirected(u, rc[u]);
    }
    return T;
}

```

常见坑:

- 普通树边必须无向建图。
- 调用 build_tree_info 前先确认 is_connected_tree(T)，否则有向边、森林、一般图都会让父子关系失真。
- tin/tout 是 DFS 序，不是 BFS 序。
- 子树区间 [tin[u], tout[u]] 只在固定根后成立。
- 换根后子树概念会改变。
- 链状树递归 DFS 可能爆栈，本模块用迭代版。

暴力/部分分替代:

- $n \leq 2000$: 每次询问直接 DFS 子树/路径。
- 只问一次子树大小: 临时 DFS 即可。
- 不会 DFS 序时，先用 children 每次遍历子树拿部分分。

升级方向:

- TreeInfo -> LCA。
- tin/tout -> 树状数组/SegmentTree 子树查询。
- children -> DP-14 树形 DP。
- parent/depth -> 暴力爬父亲 LCA。

最小测试样例:

```
5
1 2
1 3
2 4
2 5
root=1
parent[4]=2
depth[4]=2
subtree_size[2]=3
```

TREE-02 树直径、换根 DP 与树上差分

模块编号: TREE-02

模块名称: 树直径、换根 DP、树上差分

标签: [树][直径][换根 DP][树上差分][LCA][1-index]

一句话用途: 补齐树题最常见的三个进阶拼接: 求树上最远距离、每个点作为根/中心的答案、对多条路径做批量统计。

题面触发词:

- 树上最长路径、直径、最远点。
- “以每个点为根/中心” 的答案。
- 多次给树上路径加一, 最后统计每条边/每个点被经过次数。
- 树上所有点距离和。

什么时候用:

- 输入确认是一棵无向树。
- 多个子树答案需要从父亲“换根”传给儿子。
- 多次路径修改后统一统计。
- 想把树题从暴力 DFS 升级到 $O(n)$ 或 $O((n+q)\log n)$ 。

不要什么时候用:

- 一般图有环不能直接用树直径/树差分。
- 动态树加删边不是本模块。
- 路径上需要在线查询最值/修改, 可能转树链剖分, 低优先级。
- 换根 DP 的转移因题而异, 本模块给最典型的“距离和”模型。

复杂度:

- 树直径: $O(n)$ 。
- 换根求每点到所有点距离和: $O(n)$ 。
- 树上差分: $O((n+q)\log n + n)$, 瓶颈是 LCA。

数据范围参考:

- $n, q \leq 2e5$ 常规可用。
- 边权距离和可能超过 `int`, 使用 `ll`。

依赖的标准容器:

- 标准 `Graph`。
- `TreeInfo` 可辅助父亲、深度、DFS 序。
- LCA 用于树上差分。

输入如何整理:

```
Graph T(n);
for (int i = 1; i <= n - 1; i++) {
    int u, v;
    ll w;
    cin >> u >> v >> w;
    T.add_undirected(u, v, w);
}
```

接口:

```

pair<int, ll> farthest_from(const Graph& T, int s);
ll tree_diameter_length(const Graph& T);
vector<ll> sum_distance_all_roots(const Graph& T, int root);
vector<ll> vertex_path_counts(const Graph& T, const vector<pair<int,int>>& queries, const LCA& lca);
vector<ll> edge_path_counts_by_child(const Graph& T, const vector<pair<int,int>>& queries, const LCA& lca);

```

输出能力:

- 树直径长度。
- 每个点到所有点的距离和。
- 多次路径后每个点/每条父子边被经过的次数。

下游可接:

- LCA。
- TreeInfo。
- 树状数组/SegmentTree 子树查询。
- 树形 DP。

可拼接模块:

- Graph(Tree) + farthest twice。
- TreeInfo + reroot DP。
- LCA + tree difference + postorder accumulation。

模板代码:

```

pair<int, ll> farthest_from(const Graph &T, int s) {
    vector<ll> dist(T.n + 1, -1);
    queue<int> q;
    q.push(s);
    dist[s] = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (auto e : T.g[u]) {
            int v = e.to;
            if (dist[v] != -1) continue;
            dist[v] = dist[u] + e.w;
            q.push(v);
        }
    }

    int best = s;
    for (int i = 1; i <= T.n; i++) {
        if (dist[i] > dist[best]) best = i;
    }
    return {best, dist[best]};
}

ll tree_diameter_length(const Graph &T) {
    auto a = farthest_from(T, 1);
    auto b = farthest_from(T, a.first);
    return b.second;
}

vector<ll> sum_distance_all_roots(const Graph &T, int root = 1) {
    int n = T.n;
    vector<int> parent(n + 1, 0), sz(n + 1, 1), order;
    vector<ll> down(n + 1, 0), ans(n + 1, 0);

    queue<int> q;
    q.push(root);
    parent[root] = -1;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
    }
}

```

```

    order.push_back(u);
    for (auto e : T.g[u]) {
        int v = e.to;
        if (v == parent[u]) continue;
        parent[v] = u;
        q.push(v);
    }
}

for (int i = (int)order.size() - 1; i >= 0; i--) {
    int u = order[i];
    sz[u] = 1;
    down[u] = 0;
    for (auto e : T.g[u]) {
        int v = e.to;
        if (parent[v] != u) continue;
        sz[u] += sz[v];
        down[u] += down[v] + (ll)sz[v] * e.w;
    }
}

ans[root] = down[root];
for (int u : order) {
    for (auto e : T.g[u]) {
        int v = e.to;
        if (parent[v] != u) continue;
        ans[v] = ans[u] - (ll)sz[v] * e.w + (ll)(n - sz[v]) * e.w;
    }
}
return ans;
}

```

树上差分代码:

```

vector<ll> vertex_path_counts(const Graph &T, const vector<pair<int, int>> &queries, const LCA &lca) {
    vector<ll> diff(T.n + 1, 0), cnt(T.n + 1, 0);
    vector<int> parent = lca.up[0];
    vector<int> order;
    queue<int> q;
    int root = lca.root;
    q.push(root);
    vector<int> vis(T.n + 1, 0);
    vis[root] = 1;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for (auto e : T.g[u]) {
            int v = e.to;
            if (vis[v]) continue;
            vis[v] = 1;
            q.push(v);
        }
    }

    for (auto [u, v] : queries) {
        int c = lca.query(u, v);
        diff[u]++;
        diff[v]++;
        diff[c]--;
        if (c != root) diff[parent[c]]--;
    }

    for (int i = (int)order.size() - 1; i >= 0; i--) {

```

```

        int u = order[i];
        cnt[u] += diff[u];
        if (u != root) cnt[parent[u]] += cnt[u];
    }
    return cnt;
}

vector<ll> edge_path_counts_by_child(const Graph &T, const vector<pair<int, int>> &queries, const LCA &lca) {
    vector<ll> diff(T.n + 1, 0), cnt(T.n + 1, 0);
    vector<int> parent = lca.up[0];
    vector<int> order;
    queue<int> q;
    int root = lca.root;
    q.push(root);
    vector<int> vis(T.n + 1, 0);
    vis[root] = 1;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for (auto e : T.g[u]) {
            int v = e.to;
            if (vis[v]) continue;
            vis[v] = 1;
            q.push(v);
        }
    }

    for (auto [u, v] : queries) {
        int c = lca.query(u, v);
        diff[u]++;
        diff[v]++;
        diff[c] -= 2;
    }

    for (int i = (int)order.size() - 1; i >= 0; i--) {
        int u = order[i];
        cnt[u] += diff[u];
        if (u != root) cnt[parent[u]] += cnt[u];
    }
    return cnt; // 边 parent[u]-u 被经过次数是 cnt[u]
}

```

调用示例:

```

cout << tree_diameter_length(T) << '\n';

auto sumDist = sum_distance_all_roots(T, 1);
for (int u = 1; u <= n; u++) cout << sumDist[u] << '\n';

LCA lca;
lca.build(T, 1);
vector<pair<int, int>> qs = {{2, 5}, {3, 4}};
auto edgeCnt = edge_path_counts_by_child(T, qs, lca);

```

常见坑:

- 树直径在带权树上也可以两次最远点, 但边权应非负; 若有负权树, 题意通常不是普通直径。
- `sum_distance_all_roots` 的换根公式中, 边权 w 不能漏。
- 树上差分分“点被经过次数”和“边被经过次数”, 扣 LCA 的方式不同。
- `edge_path_counts_by_child` 返回的是以子节点 u 标记父子边 $\text{parent}[u]-u$ 。
- LCA 必须和差分用同一个根。
- 根节点没有父边, 不要输出 `cnt[root]` 当边答案。

暴力/部分替代:

- 直径：从每个点 DFS 求最远， $O(n^2)$ 小数据可过。
- 距离和：每个点跑一次 DFS/BFS， $O(n^2)$ 。
- 路径计数：每个询问 DFS 找路径逐点加， $O(nq)$ 。
- q 很小或 $n \leq 2000$ ，暴力路径常能拿分。

升级方向：

- 单次路径 -> DFS。
- 多次路径距离 -> LCA。
- 多次路径批量统计 -> 树上差分。
- 每点作为根的答案 -> 换根 DP。
- 子树修改查询 -> DFS 序 + 树状数组/SegmentTree。

最小测试样例：

```
5
1 2 1
1 3 1
2 4 1
2 5 1
直径长度 3，例如 4-2-1-3
路径 4-3 的边计数：4-2, 2-1, 1-3 各加 1
```

DP-14: 树形 DP

模型编号: DP-14

模型名称: 树形 DP

标签: DP、树、子树、选点、覆盖、染色

一句话用途：在树上按子树合并状态，处理选点、覆盖、染色、最大独立集等问题。

题面触发词：

- “树上选择若干点”
- “父子不能同时选”
- “覆盖所有点”
- “树上染色”
- “以子树为单位”

什么时候用：

- 输入是一棵树或森林。
- 一个节点的答案可以由儿子子树合并。
- 状态通常描述 u 自己选不选、颜色或覆盖情况。

不要什么时候用：

- 图有环且不是树，普通树形 DP 不适用。
- 问树上路径多次查询，可能是 LCA/树链/数据结构。
- 子树之间有额外横向边，状态不独立。

复杂度：

- 常见 $O(n * \text{状态数}^2)$ 或 $O(n * \text{状态数})$ 。

数据范围信号：

- $n \leq 2e5$ 且状态少：树形 DP 可做。
- 状态含背包容量：复杂度可能变成 $O(nw^2)$ 或 $O(nw)$ 。

依赖的标准容器：

- 标准 Graph G 。
- 本页后续 $\text{vector}<\text{vector}<\text{int}>> g(n + 1)$ 是速抄变体；与图论卷拼接时优先从 $G.g$ 取 $e.to/e.w$ 。
- $\text{vector}<\text{array}<\text{ll}, 2>> dp$
- $\text{vector}<\text{int}> parent, order$

输入如何整理：

- 建无向树。
- DFS 时传 parent 防止走回父亲。
- 权值统一 $w[u]$ 。

接口：

```
void dfs(int u, int p);
```

输出能力：

- 树上最大/最小选择值。
- 覆盖、染色、方案数。

下游可接：

- Graph 标准建图。
- 树上背包。
- DP-03B 状态增维：不同方向进入同一节点、parent/prev 影响未来时检查 memo key。

可拼接模块：

- Graph：统一邻接表。
- 0/1 背包：子树容量合并。
- 记忆化 DFS：状态复杂时先写。

状态句式：

$dp[u][state]$ 表示：只考虑 u 的子树，并且 u 自己处于 $state$ 状态时的最优值/方案数。

为什么这个状态够用：固定根以后，不同儿子子树之间没有横向边；父亲对子树的影响只通过 u 的状态传下来。只要 u 的状态确定，每个儿子子树就可以独立求解，再合并。

最大独立集常用：

$dp[u][0]$ 表示： u 不选时， u 子树最大权值。

$dp[u][1]$ 表示： u 选时， u 子树最大权值。

初始化：

$dp[u][0] = 0$ ：不选 u ，初始没有贡献。

$dp[u][1] = w[u]$ ：选 u ，先拿 u 的权值。

转移模板：

```
dp[u][0] += max(dp[v][0], dp[v][1]);
dp[u][1] += dp[v][0];
```

标准 Graph G 循环写法：

```
for (auto e : G.g[u]) {
    int v = e.to;
    ll w = e.w;
    if (v == p) continue;
    // 树边权不用时忽略 w
}
```

速抄局部 g 变体：

```
for (int v : g[u]) {
    if (v == p) continue;
}
```

答案位置：

- 根为 1: $\max(dp[1][0], dp[1][1])$ 。
- 森林：对每棵树答案相加。

循环顺序：

- 后序 DFS：先算儿子，再合并到父亲。
- 或先拿 DFS 序，再按逆序表推。

暴力 DFS 版本：

```

11 brute(int u, int p, int parentChosen) {
    11 ans0 = 0; // 不选 u
    for (int v : g[u]) if (v != p) ans0 += brute(v, u, 0);

    11 ans1 = -LINF;
    if (!parentChosen) {
        ans1 = w[u];
        for (int v : g[u]) if (v != p) ans1 += brute(v, u, 1);
    }
    return max(ans0, ans1);
}

```

记忆化版本:

注意: 下面 memo 版本只适用于固定根以后, u 的父亲唯一的树形 DP。若换根, 或同一个 u 可能带不同 p, 必须把 p/parent 纳入状态, 或者不要缓存。

```

vector<array<ll, 2>> memo;
vector<array<int, 2>> vis;

11 dfs_memo(int u, int p, int parentChosen) {
    if (vis[u][parentChosen]) return memo[u][parentChosen];
    vis[u][parentChosen] = 1;

    11 ans0 = 0;
    for (int v : g[u]) if (v != p) ans0 += dfs_memo(v, u, 0);

    11 ans1 = -LINF;
    if (!parentChosen) {
        ans1 = w[u];
        for (int v : g[u]) if (v != p) ans1 += dfs_memo(v, u, 1);
    }
    return memo[u][parentChosen] = max(ans0, ans1);
}

```

表推版本:

```

vector<array<ll, 2>> dp(n + 1);

void dfs(int u, int p) {
    dp[u][0] = 0;
    dp[u][1] = w[u];
    for (int v : g[u]) {
        if (v == p) continue;
        dfs(v, u);
        dp[u][0] += max(dp[v][0], dp[v][1]);
        dp[u][1] += dp[v][0];
    }
}

dfs(1, 0);
cout << max(dp[1][0], dp[1][1]) << '\n';

```

常见变体:

- 最小点覆盖: $dp[u][0] += dp[v][1]$, $dp[u][1] += \min(dp[v][0], dp[v][1])$ 。
- 树上染色: $dp[u][color]$ 合并儿子颜色。
- 树上背包: $dp[u][k]$ 表示子树选 k 个。

常见坑:

- 忘记传父亲, 递归走回去。
- 根的父亲状态处理错。
- 递归深度过大可能爆栈。
- 子树合并时更新顺序覆盖旧值。
- 固定根树形 DP 中 parent 通常是 DFS 参数; 如果同一个 u 可能从不同方向进入并被缓存, parent/prev 必须进入状态。
- n 接近 $2e5$ 时递归可能爆栈; 如果现场栈限制严格, 改用父数组 + DFS 序逆序表推更稳。

暴力/部分替代：

- $n \leq 20$ ：枚举点集检查边约束。
- 树结构明确但转移复杂：先写递归 + memo。
- 多个连通块：逐个根 DFS。

升级方向：

- 简单选点 $\rightarrow dp[u][0/1]$ 。
- 覆盖问题 $\rightarrow dp[u][0/1/2]$ 。
- 容量限制 $\rightarrow$ 树上背包。
- 换根或无固定父亲的记忆化 $\rightarrow$ 先检查 DP-03B 的 parent/prev 维度。

最小测试样例：

```
3
w: 1 2 3
edges: 1-2, 1-3
输出: 5
说明: 选 2 和 3。
```

DP-15: DAG DP

模型编号: DP-15

模型名称: DAG DP

标签: DP、拓扑排序、有向无环图、路径计数、依赖关系

一句话用途: 在有向无环图上按拓扑序转移, 求路径最优值、方案数或依赖完成方式。

题面触发词:

- “有向无环图”
- “依赖关系”
- “先修课程”
- “任务 A 必须在 B 前”
- “从起点到终点的路径数/最长路”

什么时候用:

- 状态之间是有向依赖且无环。
- 可以拓扑排序。
- $dp[u]$ 能由前驱或后继转移。

不要什么时候用:

- 图有环, 普通 DAG DP 不适用。
- 边权非负最短路在一般图上, 转 Dijkstra。
- 无权最少步数, 转 BFS。

复杂度:

- 拓扑排序 $O(n+m)$ 。
- DP 转移 $O(n+m)$ 。

数据范围信号:

- $n, m \leq 2e5$: DAG DP 很适合。
- 若 $n \leq 20$ 且集合访问, 可能是状压。

依赖的标准容器:

- 标准 Graph G。
- 本页后续 `vector<vector<pair<int,ll>>> g` 是速抄变体; 与图论卷拼接时优先使用 `G.g[u]` 中的 `e.to/e.w`。
- `vector<int> indeg, topo`
- `queue<int>`
- `vector<ll> dp`

输入如何整理:

- 建有向边 $u \rightarrow v$ 。

- 统计入度。
- 如果题意是 “u 依赖 v”，先确认边方向。

接口：

```
vector<int> topo_sort(const Graph& G);
```

输出能力：

- 到达每点的最长/最短路径。
- 路径方案数。
- 依赖完成最早时间。

下游可接：

- Graph 标准建图。
- Topo 模块。

可拼接模块：

- Graph。
- 拓扑排序。
- 取模计数。

状态句式：

$dp[u]$ 表示：到达 u 的最优值/方案数。

为什么这个状态够用：DAG 的拓扑序保证所有影响 u 的前驱都已经处理完；未来只需要知道 “到达 u 的当前最优/方案数”，不需要知道具体路径历史。

也可反向：

$dp[u]$ 表示：从 u 出发到终点的最优值/方案数。

初始化：

起点 s ： $dp[s] = 0/1$ 。

最大值题其他点为 $-LINF$ ；最小值题为 $LINF$ ；方案数为 0 。

转移模板：

```
for (auto e : G.g[u]) {
    int v = e.to;
    ll w = e.w;
    dp[v] = max(dp[v], dp[u] + w);
}
```

速抄局部 g 变体：

```
for (auto [v, w] : g[u]) {
    dp[v] = max(dp[v], dp[u] + w);
}
```

方案数：

```
ways[v] = (ways[v] + ways[u]) % MOD;
```

答案位置：

- 指定终点： $dp[t]$ 。
- 任意终点： $\max/\min dp[u]$ 。
- 路径总数：通常 $ways[t]$ 。

循环顺序：

- 先拓扑排序。
- 按拓扑序从前往后转移。
- 反向定义时按逆拓扑序。

暴力 DFS 版本：

```
ll dfs(int u) {
    if (u == t) return 0;
    ll ans = -LINF;
    for (auto [v, w] : g[u]) {
```

```

        ll sub = dfs(v);
        if (sub != -LINF) ans = max(ans, w + sub);
    }
    return ans;
}

```

记忆化版本:

```

vector<ll> memo;
vector<int> vis;

ll dfs(int u) {
    if (u == t) return 0;
    if (vis[u]) return memo[u];
    vis[u] = 1;
    ll ans = -LINF;
    for (auto [v, w] : g[u]) {
        ll sub = dfs(v);
        if (sub != -LINF) ans = max(ans, w + sub);
    }
    return memo[u] = ans;
}

```

表推版本:

```

queue<int> q;
for (int i = 1; i <= n; i++) if (indeg[i] == 0) q.push(i);
vector<int> topo;
while (!q.empty()) {
    int u = q.front(); q.pop();
    topo.push_back(u);
    for (auto [v, w] : g[u]) {
        if (--indeg[v] == 0) q.push(v);
    }
}
if ((int)topo.size() != n) {
    // 有环时普通 DAG DP 不适用; 按题意输出无解, 或重新路由到图算法。
    return;
}

```

```

vector<ll> dp(n + 1, -LINF);
dp[s] = 0;
for (int u : topo) {
    if (dp[u] == -LINF) continue;
    for (auto [v, w] : g[u]) dp[v] = max(dp[v], dp[u] + w);
}
cout << dp[t] << '\n';

```

常见变体:

- 最短路 DAG: min 转移, 初值 LINF。
- 路径计数: ways[s]=1。
- 任务最早完成时间: 边/点权表示耗时。

常见坑:

- 图有环但仍递归, 导致死循环。
- 边方向建反。
- 多个入度 0 起点没有初始化。
- 修改 indeg 后还要复用原入度, 需备份。

暴力/部分替代:

- 小图 DFS 枚举路径。
- 无环但顺序不清: 记忆化 DFS。
- 大图: 拓扑表推。

升级方向:

- DFS memo -> 拓扑 DP。
- 与 Graph 标准容器统一。
- 若有环且问最长路，通常不是普通 DP，要重新路由。

最小测试样例：

```
4 4
1 2 3
1 3 2
2 4 4
3 4 5
s=1 t=4
输出：7
```